app\sapp\streamlit_app.py

```
import sys
from pathlib import Path

import streamlit as st

ROOT = Path(__file__).resolve().parents[1]
SRC = ROOT / "src"

if str(SRC) not in sys.path:
    sys.path.insert(0, str(SRC))

from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.services.conversion_service import
ConversionService


st.set_page_config(
    page_title="IDMS Retrieval COBOL to DB2 COBOL Converter",
    layout="wide",
)

st.title("IDMS Retrieval COBOL to DB2 COBOL Converter")

st.markdown(
    """
This tool converts **IDMS retrieval COBOL** patterns into **DB2 embedded
SQL COBOL** using:

- COBOL code
- Canonical model JSON
- Phase 2 metadata JSON
- DB2 DDL
- Optional relationship overrides
- IDMS schema listing
"""
)

with st.sidebar:
```

```
    st.header("Conversion Settings")

    target_program = st.text_input(
        "Target PROGRAM-ID",
        value="",
        placeholder="Example: DEPTDB2",
    )

    use_ddl = st.checkbox(
        "Use DB2 DDL",
        value=True,
    )

    use_overrides = st.checkbox(
        "Use relationship key overrides",
        value=True,
    )

    use_idms_schema = st.checkbox(
        "Use IDMS schema listing",
        value=True,
    )

tab1, tab2, tab3, tab4, tab5, tab6 = st.tabs(
    [
        "1. COBOL Code",
        "2. Canonical Model",
        "3. Phase 2 Metadata",
        "4. DDL",
        "5. Optional Overrides",
        "6. IDMS Schema Listing",
    ]
)

with tab1:
    cobol_text = st.text_area(
        "Paste IDMS COBOL retrieval program",
        height=420,
        placeholder="Paste DEPTRPT or another IDMS retrieval program
here...",
```

```
    )

with tab2:
    canonical_json = st.text_area(
        "Paste canonical model JSON",
        height=420,
        placeholder='{ "records": [], "sets": [], "relationships": [] }',
    )

with tab3:
    phase2_metadata_json = st.text_area(
        "Paste Phase 2 conversion metadata JSON from Phase 1",
        height=420,
        placeholder='{ "version": "1.0", "field_map": {},
"navigation_intent": {}, "paragraph_operation_graph": {} }',
    )

with tab4:
    ddl_text = st.text_area(
        "Paste DB2 DDL",
        height=420,
        placeholder="CREATE TABLE DEPARTMENT (...);",
        disabled=not use_ddl,
    )

with tab5:
    overrides_json = st.text_area(
        "Paste optional relationship key overrides",
        height=300,
        placeholder="""
{
  "relationships": [
    {
      "set_name": "DEPT-EMPLOYEE",
      "parent_record": "DEPARTMENT",
      "child_record": "EMPLOYEE",
      "parent_key": "DEPT_ID",
      "child_fk": "DEPT_ID",
      "order_by": ["EMP_ID"]
    }
```

```
  ]
}
""".strip(),
        disabled=not use_overrides,
    )

with tab6:
    idms_schema_text = st.text_area(
        "Paste IDMS schema/subschema listing",
        height=420,
        placeholder="Paste SCHEMA=EMPSCHM VERSION=(100) ... RECORD
NAME........ EMPLOYEE ...",
        disabled=not use_idms_schema,
    )

convert_clicked = st.button("Convert Retrieval Program", type="primary")

if convert_clicked:
    if not cobol_text.strip():
        st.error("COBOL code is required.")
        st.stop()

    canonical_is_empty = not canonical_json.strip()
    metadata_is_empty = not phase2_metadata_json.strip()
    ddl_is_empty = not ddl_text.strip() if use_ddl else True
    idms_schema_is_empty = not idms_schema_text.strip() if use_idms_schema
else True

    if canonical_is_empty and metadata_is_empty and ddl_is_empty and
idms_schema_is_empty:
        st.error(
            "Provide at least one schema source: canonical model JSON, "
            "Phase 2 metadata JSON, DB2 DDL, or IDMS schema listing."
        )
        st.stop()

    if canonical_is_empty:
        canonical_json = '{"records": [], "relationships": []}'

    try:
```

```
        service = ConversionService()

        converted, validation_messages = service.convert_retrieval(
            cobol_text=cobol_text,
            canonical_json=canonical_json,
            phase2_metadata_json=phase2_metadata_json,
            ddl_text=ddl_text if use_ddl else None,
            idms_schema_text=idms_schema_text if use_idms_schema else
None,
            relationship_overrides_json=overrides_json if use_overrides
else None,
            target_program=target_program.strip() or None,
        )

        st.success("Conversion completed.")

        if validation_messages:
            st.warning("Conversion completed with validation messages.")
            for message in validation_messages:
                st.write(f"- {message}")
        else:
            st.info("Validation passed.")

        st.subheader("Converted DB2 COBOL")

        st.text_area(
            "Copy the converted COBOL from here",
            value=converted,
            height=650,
        )

    except ConversionError as exc:
        st.error("Conversion failed.")
        st.code(str(exc), language="text")

    except Exception as exc:
        st.error("Unexpected error.")
        st.code(str(exc), language="text")
```

src\idms_db2_converter\generators\cobol_builder.py

```python
from __future__ import annotations

from dataclasses import dataclass, field
from typing import Protocol


class CobolNode(Protocol):
    def render(self, indent: int = 7, final: bool = False) -> list[str]:
        ...


def pad(indent: int) -> str:
    return " " * indent


def ensure_period(value: str) -> str:
    value = value.rstrip()
    if value.endswith("."):
        return value
    return value + "."


@dataclass
class PreRenderedBlock:
    lines: list[str]

    def render(self, indent: int = 7, final: bool = False) -> list[str]:
        result = list(self.lines)

        if final and result:
            result[-1] = ensure_period(result[-1])

        return result


@dataclass
class RawLines:
```

```
    lines: list[str]

    def render(self, indent: int = 7, final: bool = False) -> list[str]:
        result = []

        for line in self.lines:
            if line.strip():
                result.append(pad(indent) + line.strip())
            else:
                result.append("")

        if final and result:
            result[-1] = ensure_period(result[-1])

        return result


@dataclass
class Move:
    source: str
    target: str

    def render(self, indent: int = 7, final: bool = False) -> list[str]:
        return [f"{pad(indent)}MOVE {self.source} TO {self.target}."]


@dataclass
class Perform:
    paragraph: str

    def render(self, indent: int = 7, final: bool = False) -> list[str]:
        return [f"{pad(indent)}PERFORM {self.paragraph}."]


@dataclass
class Continue:
    def render(self, indent: int = 7, final: bool = False) -> list[str]:
        return [f"{pad(indent)}CONTINUE."]
```

```
@dataclass
class ReadAtEndMove:
    file_name: str
    source: str
    target: str

    def render(self, indent: int = 7, final: bool = False) -> list[str]:
        source = self.source

        if len(source) == 1 and not source.startswith("'"):
            source = f"'{source}'"

        return [
            f"{pad(indent)}READ {self.file_name} AT END MOVE {source} TO
{self.target}."
        ]


@dataclass
class Write:
    record: str
    source: str | None = None

    def render(self, indent: int = 7, final: bool = False) -> list[str]:
        if self.source:
            return [f"{pad(indent)}WRITE {self.record} FROM
{self.source}."]

        return [f"{pad(indent)}WRITE {self.record}."]


@dataclass
class ExecSql:
    sql: str

    def render(self, indent: int = 7, final: bool = False) -> list[str]:
        normalized = self._normalize_sql_lines(self.sql)
        result = []

        for line in normalized:
```

```
            if line.strip():
                result.append(pad(indent) + line)
            else:
                result.append("")

        if result:
            result[-1] = ensure_period(result[-1])

        return result

    def _normalize_sql_lines(self, sql: str) -> list[str]:
        raw_lines = sql.splitlines()

        stripped_lines = [line.rstrip() for line in raw_lines if
line.strip()]
        if not stripped_lines:
            return []

        min_indent = min(
            len(line) - len(line.lstrip(" "))
            for line in stripped_lines
        )

        normalized = []
        for line in raw_lines:
            if line.strip():
                normalized.append(line[min_indent:].rstrip())
            else:
                normalized.append("")

        return normalized


@dataclass
class IfBlock:
    condition: str
    then_body: list[CobolNode] = field(default_factory=list)
    else_body: list[CobolNode] = field(default_factory=list)

    def render(self, indent: int = 7, final: bool = False) -> list[str]:
```

```
        result = [f"{pad(indent)}IF {self.condition}"]

        result.extend(render_nodes(self.then_body, indent + 3))

        if self.else_body:
            result.append(f"{pad(indent)}ELSE")
            result.extend(render_nodes(self.else_body, indent + 3))

        terminator = "END-IF."
        if not final:
            terminator = "END-IF"

        result.append(f"{pad(indent)}{terminator}")

        return result

    def with_prefix(self, prefix: CobolNode):
        return PrefixedBlock(prefix=prefix, block=self)


@dataclass
class PrefixedBlock:
    prefix: CobolNode
    block: CobolNode

    def render(self, indent: int = 7, final: bool = False) -> list[str]:
        result = []
        result.extend(self.prefix.render(indent=indent))
        result.append("")
        result.extend(self.block.render(indent=indent, final=final))
        return result


@dataclass
class PerformUntil:
    condition: str
    body: list[CobolNode] = field(default_factory=list)

    def render(self, indent: int = 7, final: bool = False) -> list[str]:
        result = [f"{pad(indent)}PERFORM UNTIL {self.condition}"]
```

```
        result.extend(render_nodes(self.body, indent + 3))

        terminator = "END-PERFORM."
        if not final:
            terminator = "END-PERFORM"

        result.append(f"{pad(indent)}{terminator}")

        return result


@dataclass
class Paragraph:
    name: str
    body: list[CobolNode] = field(default_factory=list)

    def render(self, indent: int = 7, final: bool = False) -> list[str]:
        result = [f"     {self.name}."]

        if self.body:
            result.extend(render_nodes(self.body, indent,
terminate_last=True))

        return result


def render_nodes(
    nodes: list[CobolNode],
    indent: int = 7,
    terminate_last: bool = False,
) -> list[str]:
    result = []

    for index, node in enumerate(nodes):
        is_last = terminate_last and index == len(nodes) - 1
        result.extend(node.render(indent=indent, final=is_last))

    return result
```

```
def render_paragraph(name: str, nodes: list[CobolNode]) -> str:
    return "\n".join(Paragraph(name=name, body=nodes).render())
```

src\idms_db2_converter\generators\cursors.py

```
from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.generators.formatting import comma_block
from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.models import SchemaModel


class CursorGenerator:
    def generate(self, schema: SchemaModel, used_sets: list[str]) -> str:
        blocks = []

        for set_name in used_sets:
            rel = schema.relationships.get(set_name)

            if not rel:
                raise ConversionError(f"Set {set_name} missing from schema
relationships.")

            if not rel.parent_key or not rel.child_fk:
                raise ConversionError(f"Set {set_name} missing parent_key
or child_fk.")

            child = schema.records.get(rel.child_record)

            if not child:
                raise ConversionError(f"Child record {rel.child_record}
missing.")

            columns = list(child.fields.keys())
            parent_hv = Naming.hv(rel.parent_record, rel.parent_key)

            lines = [
                "       EXEC SQL",
                f"            DECLARE {Naming.cursor(set_name)} CURSOR
FOR",
            ]

            lines.extend(
                comma_block(
```

```
                    items=columns,
                    first_prefix="                SELECT ",
                    next_prefix="                       ",
                )
            )

            lines.append(f"                  FROM
{self._table_name(schema, rel.child_record)}")
            lines.append(f"                 WHERE {rel.child_fk} =
:{parent_hv}")

            if rel.order_by:
                lines.append(f"                ORDER BY {',
'.join(rel.order_by)}")

            lines.append("       END-EXEC.")

            blocks.append("\n".join(lines))

        return "\n\n".join(blocks)

    def _table_name(self, schema: SchemaModel, record_name: str) -> str:
        mapped = schema.record_table_map.get(record_name)

        if mapped:
            return mapped

        return record_name.replace("-", "_").upper()
```

src\idms_db2_converter\generators\fetch_paragraphs.py

```
from idms_db2_converter.generators.sql_snippets import SqlSnippets
from idms_db2_converter.models import SchemaModel


class FetchParagraphGenerator:
    def __init__(self, schema: SchemaModel):
        self.schema = schema
        self.sql = SqlSnippets(schema)

    def paragraph_name(self, set_name: str) -> str:
        return f"FETCH-{set_name}".upper()

    def generate(self, used_sets: list[str]) -> str:
        paragraphs = []
        seen = set()

        for set_name in used_sets:
            set_name = set_name.upper()

            if set_name in seen:
                continue

            seen.add(set_name)

            if set_name not in self.schema.relationships:
                continue

            paragraphs.append(self._generate_one(set_name))

        return "\n\n".join(paragraphs)

    def _generate_one(self, set_name: str) -> str:
        paragraph_name = self.paragraph_name(set_name)
        fetch_sql = self.sql.fetch_for_set(set_name)

        return "\n".join(
            [
```

```
            f"     {paragraph_name}.",
            fetch_sql,
            "",
            "        IF SQLCODE NOT = 0 AND SQLCODE NOT = 100",
            "           PERFORM SQL-ERROR",
            "        END-IF.",
        ]
    )
```

src\idms_db2_converter\generators\formatting.py

```python
def comma_block(
    items: list[str],
    first_prefix: str,
    next_prefix: str,
) -> list[str]:
    lines = []

    for index, item in enumerate(items):
        suffix = "," if index < len(items) - 1 else ""

        if index == 0:
            lines.append(f"{first_prefix}{item}{suffix}")
        else:
            lines.append(f"{next_prefix}{item}{suffix}")

    return lines
```

src\idms_db2_converter\generators\host_variables.py

```
from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.models import SchemaModel


class HostVariableGenerator:
    def generate(self, schema: SchemaModel, used_records: list[str]) ->
str:
        lines = [
            "       EXEC SQL",
            "            INCLUDE SQLCA",
            "       END-EXEC.",
            "",
            "       EXEC SQL",
            "            BEGIN DECLARE SECTION",
            "       END-EXEC.",
            "",
        ]

        null_indicators = []
        emitted_null_indicators = set()

        for logical_record_name in used_records:
            logical_record_name = logical_record_name.upper()

            physical_record_name = self._physical_record_name(
                schema=schema,
                logical_record_name=logical_record_name,
            )

            if physical_record_name not in schema.records:
                raise ConversionError(
                    f"Record {logical_record_name} missing from schema."
                )

            record = schema.records[physical_record_name]
            lines.append(f"     01 HV-{logical_record_name}.")
```

```
            for column in record.fields.values():
                hv = Naming.hv(logical_record_name, column.name)

                lines.extend(
                    self._column_to_cobol(
                        hv=hv,
                        datatype=column.datatype,
                        length=column.length,
                        scale=column.scale,
                    )
                )

                if column.nullable:
                    ni = Naming.ni(logical_record_name, column.name)

                    if ni not in emitted_null_indicators:
                        null_indicators.append(
                            f"     01 {ni:<40} PIC S9(4) COMP."
                        )
                        emitted_null_indicators.add(ni)

            lines.append("")

        if null_indicators:
            lines.extend(null_indicators)
            lines.append("")

        lines.extend(
            [
                "       EXEC SQL",
                "            END DECLARE SECTION",
                "       END-EXEC.",
            ]
        )

        return "\n".join(lines)

    def _physical_record_name(
        self,
        schema: SchemaModel,
```

```
        logical_record_name: str,
    ) -> str:
        logical_record_name = logical_record_name.upper()

        mapped = schema.record_table_map.get(logical_record_name)

        if mapped and mapped in schema.records:
            return mapped

        if logical_record_name in schema.records:
            return logical_record_name

        normalized = logical_record_name.replace("-", "_")

        if normalized in schema.records:
            return normalized

        return logical_record_name

    def _column_to_cobol(
        self,
        hv: str,
        datatype: str,
        length: int | None,
        scale: int | None,
    ) -> list[str]:
        datatype = datatype.upper()

        if datatype == "SMALLINT":
            return [f"        05 {hv:<40} PIC S9(4) COMP."]

        if datatype in {"INTEGER", "INT"}:
            return [f"        05 {hv:<40} PIC S9(9) COMP."]

        if datatype == "BIGINT":
            return [f"        05 {hv:<40} PIC S9(18) COMP-3."]

        if datatype in {"DECIMAL", "NUMERIC"}:
            return self._decimal_to_cobol(hv, length, scale)
```

```
        if datatype in {"CHAR", "CHARACTER"}:
            return [f"        05 {hv:<40} PIC
X({self._safe_length(length)})."]

        if datatype in {"VARCHAR", "CHARACTER VARYING"}:
            return [
                f"        05 {hv:<40} PIC X({self._safe_length(length,
default=255)})."
            ]

        if datatype == "DATE":
            return [f"        05 {hv:<40} PIC X(10)."]

        if datatype == "TIMESTAMP":
            return [f"        05 {hv:<40} PIC X(26)."]

        raise ConversionError(f"Unsupported datatype: {datatype}")

    def _decimal_to_cobol(
        self,
        hv: str,
        precision: int | None,
        scale: int | None,
    ) -> list[str]:
        safe_precision = precision or 9
        safe_scale = scale or 0

        integer_digits = safe_precision - safe_scale

        if safe_scale > 0:
            if integer_digits <= 0:
                return [f"        05 {hv:<40} PIC SV9({safe_scale})
COMP-3."]

            return [
                f"        05 {hv:<40} PIC
S9({integer_digits})V9({safe_scale}) COMP-3."
            ]

        return [f"        05 {hv:<40} PIC S9({safe_precision}) COMP-3."]
```

```
    def _safe_length(
        self,
        length: int | None,
        default: int = 1,
    ) -> int:
        if length is None or length <= 0:
            return default

        return length
```

src\idms_db2_converter\generators\naming.py

```python
class Naming:
    @staticmethod
    def hv(record: str, field: str) -> str:
        return f"HV-{record}-{field}".replace("_", "-").upper()

    @staticmethod
    def ni(record: str, field: str) -> str:
        return f"NI-{record}-{field}".replace("_", "-").upper()

    @staticmethod
    def cursor(set_name: str) -> str:
        return f"C-{set_name}".upper()
```

src\idms_db2_converter\parsers\canonical_parser.py

```
import json

from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.models import Column, Record, Relationship,
SchemaModel


class CanonicalParser:
    def parse(self, text: str) -> SchemaModel:
        try:
            payload = json.loads(text)
        except json.JSONDecodeError as exc:
            raise ConversionError(f"Invalid canonical JSON: {exc}") from
exc

        schema = SchemaModel()

        self._parse_records(payload, schema)
        self._parse_relationships(payload, schema)
        self._parse_field_map(payload, schema)

        return schema

    def _parse_records(self, payload: dict, schema: SchemaModel) -> None:
        for record_json in payload.get("records", []):
            record_name = record_json["name"].upper()
            primary_key =
self._upper_or_none(record_json.get("primary_key"))

            fields = {}

            for field_json in record_json.get("fields", []):
                field_name = field_json["name"].upper()

                nullable = field_json.get("nullable")
                if nullable is None:
                    nullable = field_name != primary_key
```

```
                fields[field_name] = Column(
                    name=field_name,
                    datatype=field_json["datatype"].upper(),
                    length=field_json.get("length"),
                    scale=field_json.get("scale"),
                    nullable=bool(nullable),
                )

            schema.records[record_name] = Record(
                name=record_name,
                primary_key=primary_key,
                fields=fields,
            )

    def _parse_relationships(self, payload: dict, schema: SchemaModel) ->
None:
        for rel_json in payload.get("relationships", []):
            set_name = rel_json["set_name"].upper()
            parent_record = rel_json["parent_record"].upper()
            child_record = rel_json["child_record"].upper()

            parent_key = self._upper_or_none(rel_json.get("parent_key"))
            child_fk = self._upper_or_none(rel_json.get("child_fk"))

            if not parent_key:
                parent_key = self._resolve_parent_key(schema,
parent_record)

            if not child_fk:
                child_fk = self._resolve_child_fk(
                    schema=schema,
                    parent_record=parent_record,
                    child_record=child_record,
                )

            order_by = [
                value.upper()
                for value in rel_json.get("order_by", [])
            ]
```

```
            if not order_by and child_fk:
                order_by = [child_fk]

            schema.relationships[set_name] = Relationship(
                set_name=set_name,
                parent_record=parent_record,
                child_record=child_record,
                cardinality=rel_json.get("cardinality"),
                parent_key=parent_key,
                child_fk=child_fk,
                order_by=order_by,
            )

    def _parse_field_map(self, payload: dict, schema: SchemaModel) ->
None:
        schema.field_map = {
            key.upper(): value
            for key, value in payload.get("field_map", {}).items()
        }

    def _resolve_parent_key(
        self,
        schema: SchemaModel,
        parent_record: str,
    ) -> str | None:
        record = schema.records.get(parent_record)

        if not record:
            return None

        return record.primary_key

    def _resolve_child_fk(
        self,
        schema: SchemaModel,
        parent_record: str,
        child_record: str,
    ) -> str | None:
        parent = schema.records.get(parent_record)
        child = schema.records.get(child_record)
```

```
        if not parent or not child or not parent.primary_key:
            return None

        if parent.primary_key in child.fields:
            return parent.primary_key

        return None

    def _upper_or_none(self, value: str | None) -> str | None:
        return value.upper() if value else None
```

src\idms_db2_converter\parsers\cobol_parser.py

```
import re

from idms_db2_converter.models import CobolAnalysis, Paragraph


class CobolParser:
    PROGRAM_ID = re.compile(
        r"^\s*PROGRAM-ID\.\s+([A-Z0-9-]+)\.",
        re.IGNORECASE | re.MULTILINE,
    )

    COPY_RECORD = re.compile(
        r"^\s*(?:01\s+)?COPY\s+IDMS\s+RECORD\s+([A-Z0-9-]+)\.",
        re.IGNORECASE | re.MULTILINE,
    )

    OBTAIN_CALC = re.compile(
        r"^\s*OBTAIN\s+CALC\s+([A-Z0-9-]+)\.?\s*$",
        re.IGNORECASE | re.MULTILINE,
    )

    OBTAIN_NEXT = re.compile(

r"^\s*OBTAIN\s+NEXT\s+([A-Z0-9-]+)\s+WITHIN\s+([A-Z0-9-]+)\.?\s*$",
        re.IGNORECASE | re.MULTILINE,
    )

    OBTAIN_OWNER = re.compile(
        r"^\s*OBTAIN\s+OWNER\s+WITHIN\s+([A-Z0-9-]+)\.?\s*$",
        re.IGNORECASE | re.MULTILINE,
    )

    FIND_FIRST = re.compile(
        r"^\s*FIND\s+FIRST\s+WITHIN\s+([A-Z0-9-]+)\.?\s*$",
        re.IGNORECASE | re.MULTILINE,
    )
```

```
    PARAGRAPH_HEADER = re.compile(
        r"^ {1,7}([A-Z0-9-]+)\.\s*$",
        re.IGNORECASE | re.MULTILINE,
    )

    COMMENT_OR_DIRECTIVE = re.compile(
        r"^\s*\*",
        re.MULTILINE,
    )

    def parse(self, cobol: str) -> CobolAnalysis:
        source = self._remove_comment_lines(cobol)

        program_match = self.PROGRAM_ID.search(source)

        return CobolAnalysis(
            program_id=self._upper_or_none(program_match.group(1) if
program_match else None),
            idms_records=self._unique(self.COPY_RECORD.findall(source)),

obtain_calc_records=self._unique(self.OBTAIN_CALC.findall(source)),
            obtain_next=[
                (record.upper(), set_name.upper())
                for record, set_name in self.OBTAIN_NEXT.findall(source)
            ],

obtain_owner_sets=self._unique(self.OBTAIN_OWNER.findall(source)),
            find_first_sets=self._unique(self.FIND_FIRST.findall(source)),
        )

    def paragraphs(self, cobol: str) -> list[Paragraph]:
        source = self._remove_comment_lines(cobol)

        matches = list(self.PARAGRAPH_HEADER.finditer(source))
        paragraphs = []

        for index, match in enumerate(matches):
            name = match.group(1).upper()
            start = match.start()
```

```
            end = matches[index + 1].start() if index + 1 < len(matches)
else len(source)

            paragraphs.append(
                Paragraph(
                    name=name,
                    text=source[start:end],
                )
            )

        return paragraphs

    def _remove_comment_lines(self, cobol: str) -> str:
        lines = []

        for line in cobol.splitlines():
            stripped = line.strip()

            if not stripped:
                lines.append(line)
                continue

            if stripped.startswith("*"):
                continue

            lines.append(line)

        return "\n".join(lines)

    def _unique(self, values: list[str]) -> list[str]:
        seen = set()
        result = []

        for value in values:
            upper_value = value.upper()
            if upper_value not in seen:
                seen.add(upper_value)
                result.append(upper_value)

        return result
```

```
    def _upper_or_none(self, value: str | None) -> str | None:
        return value.upper() if value else None
```

src\idms_db2_converter\parsers\ddl_parser.py

```
import re

from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.models import Column, Record, Relationship,
SchemaModel


class DDLParser:
    CREATE_TABLE = re.compile(
        r"CREATE\s+TABLE\s+((?:[A-Z0-9_]+\.)?[A-Z0-9_]+)\s*$(.*?)$\s*;",
        re.IGNORECASE | re.DOTALL,
    )

    ALTER_TABLE_FK = re.compile(
        r"""
        ALTER\s+TABLE\s+((?:[A-Z0-9_]+\.)?[A-Z0-9_]+)
        \s+ADD\s+CONSTRAINT\s+([A-Z0-9_]+)
        \s+FOREIGN\s+KEY\s*$(.*?)$
        \s+REFERENCES\s+((?:[A-Z0-9_]+\.)?[A-Z0-9_]+)\s*$(.*?)$
        """,
        re.IGNORECASE | re.DOTALL | re.VERBOSE,
    )

    TABLE_PRIMARY_KEY = re.compile(
        r"\bPRIMARY\s+KEY\s*$(.*?)$",
        re.IGNORECASE | re.DOTALL,
    )

    TABLE_FOREIGN_KEY = re.compile(
        r"""
        (?:CONSTRAINT\s+([A-Z0-9_]+)\s+)?
        FOREIGN\s+KEY\s*$(.*?)$
        \s+REFERENCES\s+((?:[A-Z0-9_]+\.)?[A-Z0-9_]+)\s*$(.*?)$
        """,
        re.IGNORECASE | re.DOTALL | re.VERBOSE,
    )
```

```
    INLINE_PRIMARY_KEY = re.compile(
        r"\bPRIMARY\s+KEY\b",
        re.IGNORECASE,
    )

    INLINE_NOT_NULL = re.compile(
        r"\bNOT\s+NULL\b",
        re.IGNORECASE,
    )

    CONSTRAINT_STARTERS = {
        "CONSTRAINT",
        "PRIMARY",
        "FOREIGN",
        "UNIQUE",
        "CHECK",
    }

    def parse(self, ddl: str) -> SchemaModel:
        schema = SchemaModel()
        schema.schema_source = "DDL"

        ddl = self._remove_comments(ddl)

        for match in self.CREATE_TABLE.finditer(ddl):
            table_name = self._normalize_name(match.group(1))
            body = match.group(2)

            record = Record(
                name=table_name,
                primary_key=None,
                fields={},
            )

            schema.record_table_map[table_name] = table_name

            entries = self._split_entries(body)

            pending_foreign_keys = []
```

```
            for entry in entries:
                entry = entry.strip()

                if not entry:
                    continue

                first_word = self._first_word(entry)

                if first_word in self.CONSTRAINT_STARTERS:
                    self._parse_table_constraint(
                        schema=schema,
                        record=record,
                        table_name=table_name,
                        entry=entry,
                        pending_foreign_keys=pending_foreign_keys,
                    )
                    continue

                column = self._parse_column(entry)

                if not column:
                    continue

                record.fields[column.name] = column

                if self.INLINE_PRIMARY_KEY.search(entry):
                    record.primary_key = column.name
                    column.nullable = False

            schema.records[table_name] = record

            for fk in pending_foreign_keys:
                self._add_relationship(
                    schema=schema,
                    child_table=table_name,
                    child_fk=fk["child_fk"],
                    parent_table=fk["parent_table"],
                    parent_key=fk["parent_key"],
                )
```

```
        self._parse_alter_table_foreign_keys(ddl, schema)
        self._validate(schema)

        return schema

    def _parse_table_constraint(
        self,
        schema: SchemaModel,
        record: Record,
        table_name: str,
        entry: str,
        pending_foreign_keys: list[dict],
    ) -> None:
        pk_match = self.TABLE_PRIMARY_KEY.search(entry)

        if pk_match:
            primary_key = self._first_column(pk_match.group(1))

            if primary_key:
                record.primary_key = primary_key

            return

        fk_match = self.TABLE_FOREIGN_KEY.search(entry)

        if fk_match:
            child_fk = self._first_column(fk_match.group(2))
            parent_table = self._normalize_name(fk_match.group(3))
            parent_key = self._first_column(fk_match.group(4))

            if child_fk and parent_table and parent_key:
                pending_foreign_keys.append(
                    {
                        "child_fk": child_fk,
                        "parent_table": parent_table,
                        "parent_key": parent_key,
                    }
                )

            return
```

```
    def _parse_alter_table_foreign_keys(
        self,
        ddl: str,
        schema: SchemaModel,
    ) -> None:
        for match in self.ALTER_TABLE_FK.finditer(ddl):
            child_table = self._normalize_name(match.group(1))
            child_fk = self._first_column(match.group(3))
            parent_table = self._normalize_name(match.group(4))
            parent_key = self._first_column(match.group(5))

            if not child_table or not child_fk or not parent_table or not
parent_key:
                continue

            self._add_relationship(
                schema=schema,
                child_table=child_table,
                child_fk=child_fk,
                parent_table=parent_table,
                parent_key=parent_key,
            )

    def _add_relationship(
        self,
        schema: SchemaModel,
        child_table: str,
        child_fk: str,
        parent_table: str,
        parent_key: str,
    ) -> None:
        child_table = child_table.upper()
        child_fk = child_fk.upper()
        parent_table = parent_table.upper()
        parent_key = parent_key.upper()

        set_name = f"{parent_table}-{child_table}"

        schema.relationships[set_name] = Relationship(
```

```
            set_name=set_name,
            parent_record=parent_table,
            child_record=child_table,
            cardinality="1:N",
            parent_key=parent_key,
            child_fk=child_fk,
            order_by=[child_fk],
        )

    def _parse_column(self, entry: str) -> Column | None:
        tokens = entry.strip().split()

        if len(tokens) < 2:
            return None

        column_name = self._clean_identifier(tokens[0])

        if not column_name:
            return None

        datatype_text = " ".join(tokens[1:])
        datatype, length, scale = self._parse_datatype(datatype_text)

        if not datatype:
            return None

        nullable = not bool(self.INLINE_NOT_NULL.search(entry))

        return Column(
            name=column_name,
            datatype=datatype,
            length=length,
            scale=scale,
            nullable=nullable,
        )

    def _parse_datatype(
        self,
        datatype_text: str,
    ) -> tuple[str | None, int | None, int | None]:
```

```
        text = datatype_text.upper().strip()

        text = re.sub(
            r"\bGENERATED\s+ALWAYS\s+AS\s+IDENTITY\b.*",
            "",
            text,
            flags=re.IGNORECASE | re.DOTALL,
        )

        text = re.sub(
            r"\bNOT\s+NULL\b.*",
            "",
            text,
            flags=re.IGNORECASE | re.DOTALL,
        )

        text = re.sub(
            r"\bPRIMARY\s+KEY\b.*",
            "",
            text,
            flags=re.IGNORECASE | re.DOTALL,
        )

        text = text.strip()

        decimal_match = re.match(
            r"(DECIMAL|NUMERIC|DEC)\s*$\s*(\d+)\s*,\s*(\d+)\s*$",
            text,
            flags=re.IGNORECASE,
        )

        if decimal_match:
            return (
                "DECIMAL",
                int(decimal_match.group(2)),
                int(decimal_match.group(3)),
            )

        char_match = re.match(
            r"(CHAR|CHARACTER)\s*$\s*(\d+)\s*$",
```

```
            text,
            flags=re.IGNORECASE,
        )

        if char_match:
            return (
                "CHAR",
                int(char_match.group(2)),
                None,
            )

        varchar_match = re.match(
            r"(VARCHAR|CHARACTER\s+VARYING)\s*$\s*(\d+)\s*$",
            text,
            flags=re.IGNORECASE,
        )

        if varchar_match:
            return (
                "VARCHAR",
                int(varchar_match.group(2)),
                None,
            )

        integer_match = re.match(
            r"(INTEGER|INT)\b",
            text,
            flags=re.IGNORECASE,
        )

        if integer_match:
            return (
                "INTEGER",
                None,
                None,
            )

        smallint_match = re.match(
            r"SMALLINT\b",
            text,
```

```
            flags=re.IGNORECASE,
        )

        if smallint_match:
            return (
                "SMALLINT",
                None,
                None,
            )

        bigint_match = re.match(
            r"BIGINT\b",
            text,
            flags=re.IGNORECASE,
        )

        if bigint_match:
            return (
                "BIGINT",
                None,
                None,
            )

        date_match = re.match(
            r"DATE\b",
            text,
            flags=re.IGNORECASE,
        )

        if date_match:
            return (
                "DATE",
                None,
                None,
            )

        timestamp_match = re.match(
            r"TIMESTAMP\b",
            text,
            flags=re.IGNORECASE,
```

```
        )

        if timestamp_match:
            return (
                "TIMESTAMP",
                None,
                None,
            )

        return None, None, None

    def _split_entries(self, body: str) -> list[str]:
        entries = []
        current = []
        depth = 0
        in_string = False
        index = 0

        while index < len(body):
            char = body[index]

            if char == "'":
                in_string = not in_string
                current.append(char)
                index += 1
                continue

            if not in_string:
                if char == "(":
                    depth += 1
                elif char == ")":
                    depth -= 1

                if char == "," and depth == 0:
                    item = "".join(current).strip()

                    if item:
                        entries.append(item)

                    current = []
```

```
                    index += 1
                    continue

            current.append(char)
            index += 1

        item = "".join(current).strip()

        if item:
            entries.append(item)

        return entries

    def _remove_comments(self, ddl: str) -> str:
        ddl = re.sub(
            r"--.*?$",
            "",
            ddl,
            flags=re.MULTILINE,
        )

        ddl = re.sub(
            r"/\*.*?\*/",
            "",
            ddl,
            flags=re.DOTALL,
        )

        return ddl

    def _validate(self, schema: SchemaModel) -> None:
        errors = []

        for record in schema.records.values():
            if record.primary_key and record.primary_key not in
record.fields:
                errors.append(
                    f"{record.name} primary key {record.primary_key} is
not declared as a column."
                )
```

```
            if record.primary_key and record.primary_key in record.fields:
                record.fields[record.primary_key].nullable = False

        if errors:
            raise ConversionError("\n".join(errors))

    def _normalize_name(self, value: str) -> str:
        value = value.strip().upper()

        if "." in value:
            value = value.split(".")[-1]

        return self._clean_identifier(value)

    def _clean_identifier(self, value: str) -> str:
        return value.strip().strip('"').strip().upper()

    def _first_word(self, value: str) -> str:
        parts = value.strip().split()

        if not parts:
            return ""

        return parts[0].upper()

    def _first_column(self, value: str) -> str | None:
        columns = [
            self._clean_identifier(item)
            for item in value.split(",")
            if item.strip()
        ]

        if not columns:
            return None

        return columns[0].upper()
```

src\idms_db2_converter\parsers\idms_schema_parser.py

```
import re

from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.models import Column, Record, Relationship,
SchemaModel


class IdmsSchemaParser:
    RECORD_HEADER = re.compile(
        r"^RECORD NAME\.+\s+([A-Z0-9-]+)",
        re.IGNORECASE | re.MULTILINE,
    )

    LOCATION_CALC = re.compile(
        r"LOCATION MODE\.+\s+CALC USING\s+([A-Z0-9-]+)",
        re.IGNORECASE,
    )

    DBKEY_SET_ROLE = re.compile(
        r"^\s+([A-Z0-9-]+)\s+(?:INDEX\s+)?(OWNER|MEMBER)\b",
        re.IGNORECASE | re.MULTILINE,
    )

    DATA_LINE_PREFIX = re.compile(
        r"^\s*(0[2-5])\s+([A-Z0-9-]+)\s+(.*)$",
        re.IGNORECASE,
    )

    PICTURE_FIND = re.compile(
        r"S?9$\d+$V9$\d+$|"
        r"S?9$\d+$V9+|"
        r"S?9+V9$\d+$|"
        r"S?9+V9+|"
        r"S?V9$\d+$|"
        r"S?V9+|"
        r"S?9$\d+$|"
        r"S?9+|"
        r"X$\d+$|"
```

```
        r"X+",
        re.IGNORECASE,
    )

    DATE_PARTS = {
        "YEAR": {
            "substring_start": 1,
            "substring_length": 4,
        },
        "MONTH": {
            "substring_start": 5,
            "substring_length": 2,
        },
        "DAY": {
            "substring_start": 7,
            "substring_length": 2,
        },
    }

    def parse(self, text: str) -> SchemaModel:
        schema = SchemaModel()
        schema.schema_source = "IDMS_SCHEMA"

        blocks = self._record_blocks(text)
        set_roles: dict[str, dict[str, str]] = {}

        for record_name, block in blocks:
            self._parse_record(schema, record_name, block)
            self._collect_set_roles(set_roles, record_name, block)

        self._build_relationships(schema, set_roles)

        if hasattr(schema, "add_validation_message"):
            schema.add_validation_message("Schema built from IDMS schema
listing.")

        return schema

    def _record_blocks(self, text: str) -> list[tuple[str, str]]:
        matches = list(self.RECORD_HEADER.finditer(text))
```

```
        result = []

        for index, match in enumerate(matches):
            record_name = match.group(1).upper()
            start = match.start()
            end = matches[index + 1].start() if index + 1 < len(matches)
else len(text)

            result.append((record_name, text[start:end]))

        return result

    def _parse_record(
        self,
        schema: SchemaModel,
        record_name: str,
        block: str,
    ) -> None:
        primary_key = None

        calc_match = self.LOCATION_CALC.search(block)

        if calc_match:
            primary_key = self._normalize_column(calc_match.group(1))

        record = Record(
            name=record_name,
            primary_key=primary_key,
            fields={},
        )

        schema.record_table_map[record_name] =
self._normalize_table(record_name)

        active_date_groups: dict[int, dict[str, str]] = {}

        for line in block.splitlines():
            parsed = self._parse_data_item_line(line)

            if not parsed:
```

```
                continue

            level = parsed["level"]
            raw_field_name = parsed["field_name"]
            body = parsed["body"]
            storage_length = parsed["length"]

            self._close_inactive_date_groups(active_date_groups, level)

            if raw_field_name == "FILLER":
                continue

            usage = self._extract_usage(body)
            picture = self._extract_picture(body)
            column_name = self._normalize_column(raw_field_name)

            if self._is_date_group(raw_field_name, picture,
storage_length):
                if column_name not in record.fields:
                    record.fields[column_name] = Column(
                        name=column_name,
                        datatype="CHAR",
                        length=8,
                        scale=None,
                        nullable=False,
                    )

                host = Naming.hv(record_name, column_name)
                self._add_field_map(schema, raw_field_name, host)

                active_date_groups[level] = {
                    "raw_field_name": raw_field_name,
                    "column_name": column_name,
                    "host": host,
                }

                continue

            date_group = self._nearest_date_group(active_date_groups)
```

```
            if date_group and self._is_date_part(raw_field_name):
                self._add_date_part_map(
                    schema=schema,
                    idms_field=raw_field_name,
                    host=date_group["host"],
                )
                continue

            if not picture:
                continue

            datatype, column_length, scale = self._map_picture(
                picture=picture,
                usage=usage,
                storage_length=storage_length,
            )

            if column_name not in record.fields:
                record.fields[column_name] = Column(
                    name=column_name,
                    datatype=datatype,
                    length=column_length,
                    scale=scale,
                    nullable=False,
                )

            self._add_field_map(
                schema=schema,
                idms_field=raw_field_name,
                host=Naming.hv(record_name, column_name),
            )

        if primary_key and primary_key not in record.fields:
            record.fields[primary_key] = Column(
                name=primary_key,
                datatype="CHAR",
                length=12,
                scale=None,
                nullable=False,
            )
```

```
        schema.records[record_name] = record

    def _parse_data_item_line(self, line: str) -> dict | None:
        match = self.DATA_LINE_PREFIX.match(line)

        if not match:
            return None

        level = int(match.group(1))
        field_name = match.group(2).upper()
        remainder = match.group(3).strip()

        if field_name == "FILLER":
            return None

        parts = remainder.split()

        if len(parts) < 2:
            return None

        if not parts[-1].isdigit() or not parts[-2].isdigit():
            return None

        length = int(parts[-1])
        start = int(parts[-2])
        body = " ".join(parts[:-2])

        return {
            "level": level,
            "field_name": field_name,
            "body": body,
            "start": start,
            "length": length,
        }

    def _collect_set_roles(
        self,
        set_roles: dict[str, dict[str, str]],
        record_name: str,
```

```
        block: str,
    ) -> None:
        for match in self.DBKEY_SET_ROLE.finditer(block):
            set_name = match.group(1).upper()
            role = match.group(2).upper()

            if set_name == "CALC":
                continue

            set_roles.setdefault(set_name, {})

            if role == "OWNER":
                set_roles[set_name]["owner"] = record_name

            if role == "MEMBER":
                set_roles[set_name]["member"] = record_name

    def _build_relationships(
        self,
        schema: SchemaModel,
        set_roles: dict[str, dict[str, str]],
    ) -> None:
        for set_name, roles in set_roles.items():
            owner = roles.get("owner")
            member = roles.get("member")

            if not owner or not member:
                continue

            if owner not in schema.records or member not in
schema.records:
                continue

            owner_record = schema.records[owner]
            member_record = schema.records[member]

            parent_key = owner_record.primary_key

            if not parent_key:
                continue
```

```
            child_fk = parent_key

            if child_fk not in member_record.fields:
                parent_column = owner_record.fields.get(parent_key)

                member_record.fields[child_fk] = Column(
                    name=child_fk,
                    datatype=parent_column.datatype if parent_column else
"CHAR",
                    length=parent_column.length if parent_column else 12,
                    scale=parent_column.scale if parent_column else None,
                    nullable=True,
                )

            schema.relationships[set_name] = Relationship(
                set_name=set_name,
                parent_record=owner,
                child_record=member,
                cardinality="1:N",
                parent_key=parent_key,
                child_fk=child_fk,
                order_by=[child_fk],
            )

    def _extract_usage(self, body: str) -> str:
        upper = body.upper()

        if "COMP-3" in upper:
            return "COMP-3"

        if re.search(r"\bCOMP\b", upper):
            return "COMP"

        return "DISPLAY"

    def _extract_picture(self, body: str) -> str | None:
        upper = body.upper()
        matches = list(self.PICTURE_FIND.finditer(upper))
```

```
        if not matches:
            return None

        valid_matches = []

        for match in matches:
            value = match.group(0).upper()
            start = match.start()
            end = match.end()

            before = upper[start - 1] if start > 0 else " "
            after = upper[end] if end < len(upper) else " "

            if before.isalnum() or before in {"-", "_"}:
                continue

            if after.isalnum() or after in {"-", "_"}:
                continue

            valid_matches.append(value)

        if not valid_matches:
            return None

        return valid_matches[-1]

    def _map_picture(
        self,
        picture: str,
        usage: str,
        storage_length: int,
    ) -> tuple[str, int | None, int | None]:
        picture = picture.upper()
        usage = usage.upper()

        if picture.startswith("X"):
            length_match = re.search(r"X$(\d+)$", picture)

            if length_match:
                return "CHAR", int(length_match.group(1)), None
```

```
            return "CHAR", len(picture), None

        if "9" in picture:
            precision = self._numeric_precision(picture)
            scale = self._numeric_scale(picture)

            if usage == "DISPLAY":
                return "CHAR", storage_length, None

            if usage == "COMP":
                return "INTEGER", None, None

            return "DECIMAL", precision, scale

        return "CHAR", storage_length, None

    def _numeric_precision(self, picture: str) -> int:
        picture = picture.upper().replace("S", "")

        if "V" in picture:
            before_v, after_v = picture.split("V", 1)
        else:
            before_v = picture
            after_v = ""

        return self._count_9_digits(before_v) +
self._count_9_digits(after_v)

    def _numeric_scale(self, picture: str) -> int:
        picture = picture.upper().replace("S", "")

        if "V" not in picture:
            return 0

        after_v = picture.split("V", 1)[1]

        return self._count_9_digits(after_v)

    def _count_9_digits(self, value: str) -> int:
```

```
        total = 0

        for match in re.finditer(r"9$(\d+)$|9", value):
            if match.group(1):
                total += int(match.group(1))
            else:
                total += 1

        return total

    def _is_date_group(
        self,
        raw_field_name: str,
        picture: str | None,
        storage_length: int,
    ) -> bool:
        normalized = self._normalize_column(raw_field_name)

        return (
            picture is None
            and normalized.endswith("DATE")
            and storage_length == 8
        )

    def _is_date_part(self, raw_field_name: str) -> bool:
        normalized = self._normalize_column(raw_field_name)

        return (
            normalized.endswith("_YEAR")
            or normalized.endswith("_MONTH")
            or normalized.endswith("_DAY")
        )

    def _add_date_part_map(
        self,
        schema: SchemaModel,
        idms_field: str,
        host: str,
    ) -> None:
        normalized = self._normalize_column(idms_field)
```

```
        if normalized.endswith("_YEAR"):
            part = "YEAR"
        elif normalized.endswith("_MONTH"):
            part = "MONTH"
        elif normalized.endswith("_DAY"):
            part = "DAY"
        else:
            return

        schema.date_part_map[idms_field.upper()] = {
            "host": host,
            "substring_start": self.DATE_PARTS[part]["substring_start"],
            "substring_length": self.DATE_PARTS[part]["substring_length"],
        }

    def _add_field_map(
        self,
        schema: SchemaModel,
        idms_field: str,
        host: str,
    ) -> None:
        schema.field_map[idms_field.upper()] = {
            "host": host,
        }

    def _close_inactive_date_groups(
        self,
        active_date_groups: dict[int, dict[str, str]],
        current_level: int,
    ) -> None:
        closed_levels = [
            level
            for level in active_date_groups
            if level >= current_level
        ]

        for level in closed_levels:
            del active_date_groups[level]
```

```
    def _nearest_date_group(
        self,
        active_date_groups: dict[int, dict[str, str]],
    ) -> dict[str, str] | None:
        if not active_date_groups:
            return None

        nearest_level = max(active_date_groups.keys())

        return active_date_groups[nearest_level]

    def _normalize_table(self, record_name: str) -> str:
        return record_name.upper().replace("-", "_")

    def _normalize_column(self, field_name: str) -> str:
        value = field_name.upper()
        value = re.sub(r"-\d{4}$", "", value)

        return value.replace("-", "_")
```

src\idms_db2_converter\parsers\phase2_metadata_parser.py

```
import json

from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.models import SchemaModel
from idms_db2_converter.models import Column, Record, Relationship,
SchemaModel


class Phase2MetadataParser:
    def parse_into_schema(self, text: str, schema: SchemaModel) ->
SchemaModel:
        if not text or not text.strip():
            return schema

        try:
            payload = json.loads(text)
        except json.JSONDecodeError as exc:
            raise ConversionError(f"Invalid Phase 2 metadata JSON: {exc}")
from exc

        schema.record_table_map = self._upper_dict_values(
            payload.get("record_table_map", {})
        )

        schema.field_map.update(
            {
                key.upper(): value
                for key, value in payload.get("field_map", {}).items()
            }
        )

        schema.calc_key_map = {
            key.upper(): value
            for key, value in payload.get("calc_key_map", {}).items()
        }

        schema.set_ordering_map = {
            key.upper(): value
```

```
            for key, value in payload.get("set_ordering_map", {}).items()
        }

        schema.navigation_intent = {
            key.upper(): value
            for key, value in payload.get("navigation_intent", {}).items()
        }

        schema.nullable_fk_map = {
            key.upper(): value
            for key, value in payload.get("nullable_fk_map", {}).items()
        }

        schema.date_part_map = {
            key.upper(): value
            for key, value in payload.get("date_part_map", {}).items()
        }

        schema.output_semantics = payload.get("output_semantics", {})

        schema.paragraph_operation_graph = {
            key.upper(): value
            for key, value in payload.get("paragraph_operation_graph",
{}).items()
        }

        self._apply_set_ordering(schema)

        return schema

    def _upper_dict_values(self, source: dict) -> dict:
        result = {}

        for key, value in source.items():
            result[key.upper()] = value.upper() if isinstance(value, str)
else value

        return result

    def _apply_set_ordering(self, schema: SchemaModel) -> None:
```

```
        for set_name, ordering in schema.set_ordering_map.items():
            relationship = schema.relationships.get(set_name)

            if not relationship:
                continue

            order_by = ordering.get("order_by", [])

            if order_by:
                relationship.order_by = [item.upper() for item in
order_by]

    def parse_as_schema(self, text: str) -> SchemaModel:
        schema = SchemaModel()

        if not text or not text.strip():
            return schema

        try:
            payload = json.loads(text)
        except json.JSONDecodeError as exc:
            raise ConversionError(f"Invalid Phase 2 metadata JSON: {exc}")
from exc

        self._parse_metadata_records(payload, schema)
        self._parse_metadata_relationships(payload, schema)

        self.parse_into_schema(text=text, schema=schema)

        schema.schema_source = "PHASE2_METADATA"

        return schema

    def _parse_metadata_records(self, payload: dict, schema: SchemaModel)
-> None:
        records_payload = (
            payload.get("records")
            or payload.get("tables")
            or payload.get("db2_tables")
            or []
```

```
        )

        for record_json in records_payload:
            record_name = (
                record_json.get("name")
                or record_json.get("record")
                or record_json.get("table")
            )

            if not record_name:
                continue

            record_name = record_name.upper()
            primary_key = self._upper_or_none(
                record_json.get("primary_key")
                or record_json.get("pk")
            )

            fields = {}

            field_payload = (
                record_json.get("fields")
                or record_json.get("columns")
                or []
            )

            for field_json in field_payload:
                field_name = (
                    field_json.get("name")
                    or field_json.get("column")
                )

                if not field_name:
                    continue

                field_name = field_name.upper()

                datatype = (
                    field_json.get("datatype")
                    or field_json.get("type")
```

```
                    or "CHAR"
                ).upper()

                nullable = field_json.get("nullable")

                if nullable is None:
                    nullable = field_name != primary_key

                fields[field_name] = Column(
                    name=field_name,
                    datatype=datatype,
                    length=field_json.get("length"),
                    scale=field_json.get("scale"),
                    nullable=bool(nullable),
                )

            schema.records[record_name] = Record(
                name=record_name,
                primary_key=primary_key,
                fields=fields,
            )

    def _parse_metadata_relationships(self, payload: dict, schema:
SchemaModel) -> None:
        relationships_payload = (
            payload.get("relationships")
            or payload.get("sets")
            or payload.get("db2_relationships")
            or []
        )

        for rel_json in relationships_payload:
            set_name = (
                rel_json.get("set_name")
                or rel_json.get("name")
                or rel_json.get("relationship")
            )

            parent_record = (
                rel_json.get("parent_record")
```

```
                or rel_json.get("parent_table")
            )

            child_record = (
                rel_json.get("child_record")
                or rel_json.get("child_table")
            )

            if not set_name or not parent_record or not child_record:
                continue

            set_name = set_name.upper()
            parent_record = parent_record.upper()
            child_record = child_record.upper()

            parent_key = self._upper_or_none(
                rel_json.get("parent_key")
                or rel_json.get("parent_pk")
            )

            child_fk = self._upper_or_none(
                rel_json.get("child_fk")
                or rel_json.get("foreign_key")
            )

            order_by = [
                value.upper()
                for value in rel_json.get("order_by", [])
            ]

            if not order_by and child_fk:
                order_by = [child_fk]

            schema.relationships[set_name] = Relationship(
                set_name=set_name,
                parent_record=parent_record,
                child_record=child_record,
                cardinality=rel_json.get("cardinality", "1:N"),
                parent_key=parent_key,
                child_fk=child_fk,
```

```
                order_by=order_by,
            )
```

src\idms_db2_converter\services\conversion_service.py

```python
from copy import deepcopy

from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.models import SchemaModel
from idms_db2_converter.parsers.cobol_parser import CobolParser
from idms_db2_converter.parsers.canonical_parser import CanonicalParser
from idms_db2_converter.parsers.ddl_parser import DDLParser
from idms_db2_converter.parsers.phase2_metadata_parser import
Phase2MetadataParser
from idms_db2_converter.parsers.idms_schema_parser import IdmsSchemaParser
from idms_db2_converter.services.relationship_resolver import
RelationshipResolver
from idms_db2_converter.transformers.retrieval_transformer import
RetrievalTransformer
from idms_db2_converter.validators.output_validator import OutputValidator
from idms_db2_converter.validators.schema_validator import SchemaValidator
from idms_db2_converter.services.schema_merger import SchemaMerger

class ConversionService:
    def convert_retrieval(
        self,
        cobol_text: str,
        canonical_json: str,
        phase2_metadata_json: str | None = None,
        ddl_text: str | None = None,
        idms_schema_text: str | None = None,
        relationship_overrides_json: str | None = None,
        target_program: str | None = None,
    ) -> tuple[str, list[str]]:
        analysis = CobolParser().parse(cobol_text)

        schema = self._build_schema_from_available_sources(
            canonical_json=canonical_json,
            phase2_metadata_json=phase2_metadata_json,
            ddl_text=ddl_text,
            idms_schema_text=idms_schema_text,
        )
```

```
        print("FINAL SCHEMA SOURCE:", getattr(schema, "schema_source",
None))

        for record_name in ["DEPARTMENT", "EMPLOYEE", "EMPOSITION", "JOB",
"OFFICE"]:
            record = schema.records.get(record_name)

            if not record:
                print("MISSING RECORD:", record_name)
                continue

            print("RECORD:", record_name)

            for field_name, field in record.fields.items():
                print(
                    "  FIELD:",
                    field_name,
                    field.datatype,
                    field.length,
                    field.scale,
                    "NULLABLE=",
                    field.nullable,
                )

        print("FIELD MAP DEPT-NAME-0410:",
schema.field_map.get("DEPT-NAME-0410"))
        print("FIELD MAP EMP-FIRST-NAME-0415:",
schema.field_map.get("EMP-FIRST-NAME-0415"))
        print("FIELD MAP EMP-LAST-NAME-0415:",
schema.field_map.get("EMP-LAST-NAME-0415"))
        print("FIELD MAP TITLE-0440:", schema.field_map.get("TITLE-0440"))
        print("FIELD MAP OFFICE-STREET-0450:",
schema.field_map.get("OFFICE-STREET-0450"))

        if relationship_overrides_json and
relationship_overrides_json.strip():
            schema = RelationshipResolver().apply_overrides(
                schema=schema,
                relationship_overrides_json=relationship_overrides_json,
            )
```

```
        schema = RelationshipResolver().resolve_cobol_sets(
            schema=schema,
            analysis=analysis,
        )

        schema_errors = SchemaValidator().validate(schema, analysis)

        if schema_errors:
            raise ConversionError("\n".join(schema_errors))

        converted = RetrievalTransformer(schema, analysis).transform(
            cobol=cobol_text,
            target_program=target_program,
        )

        output_errors = OutputValidator().validate(converted)

        validation_messages = []
        validation_messages.extend(getattr(schema, "validation_messages",
[]))
        validation_messages.extend(output_errors)

        return converted, validation_messages

    def _build_schema_from_available_sources(
        self,
        canonical_json: str | None,
        phase2_metadata_json: str | None,
        ddl_text: str | None,
        idms_schema_text: str | None,
    ) -> SchemaModel:
        canonical_has_text = bool(canonical_json and
canonical_json.strip())
        metadata_has_text = bool(phase2_metadata_json and
phase2_metadata_json.strip())
        ddl_has_text = bool(ddl_text and ddl_text.strip())
        idms_schema_has_text = bool(idms_schema_text and
idms_schema_text.strip())
```

```
        canonical_schema = SchemaModel()
        metadata_schema = SchemaModel()
        ddl_schema = SchemaModel()
        idms_schema = SchemaModel()

        if canonical_has_text:
            canonical_schema = CanonicalParser().parse(canonical_json)
            canonical_schema.schema_source = "CANONICAL"

        if metadata_has_text:
            metadata_schema = Phase2MetadataParser().parse_as_schema(
                phase2_metadata_json
            )
            metadata_schema.schema_source = "PHASE2_METADATA"

        if ddl_has_text:
            ddl_schema = DDLParser().parse(ddl_text)

            if self._has_schema_objects(ddl_schema):
                ddl_schema.schema_source = "DDL"
            elif self._looks_like_idms_schema(ddl_text):
                idms_schema = IdmsSchemaParser().parse(ddl_text)
                idms_schema.schema_source = "IDMS_SCHEMA"

        if idms_schema_has_text:
            idms_schema = IdmsSchemaParser().parse(idms_schema_text)
            idms_schema.schema_source = "IDMS_SCHEMA"

        #
----------------------------------------------------------------
        # IMPORTANT:
        # If DDL exists, DDL must be the physical base.
        # Canonical and IDMS must not override DDL physical data
types/lengths.
        #
----------------------------------------------------------------
        if self._has_schema_objects(ddl_schema):
            base_schema = ddl_schema
            base_schema.schema_source = "DDL"
```

```
            if self._has_schema_objects(canonical_schema):
                base_schema = self._merge_non_physical_metadata(
                    physical_schema=base_schema,
                    overlay=canonical_schema,
                )

            if metadata_has_text:
                base_schema = Phase2MetadataParser().parse_into_schema(
                    text=phase2_metadata_json,
                    schema=base_schema,
                )

            if self._has_schema_objects(idms_schema):
                base_schema = SchemaMerger().merge_physical_with_idms(
                    physical_schema=base_schema,
                    idms_schema=idms_schema,
                )

            return base_schema

        # If no DDL, canonical can be base.
        if self._has_schema_objects(canonical_schema):
            base_schema = canonical_schema

            if metadata_has_text:
                base_schema = Phase2MetadataParser().parse_into_schema(
                    text=phase2_metadata_json,
                    schema=base_schema,
                )

            if self._has_schema_objects(idms_schema):
                base_schema = self._merge_schema(base_schema, idms_schema)

            return base_schema

        # If no DDL and no canonical, metadata can be base.
        if self._has_schema_objects(metadata_schema):
            base_schema = metadata_schema

            if self._has_schema_objects(idms_schema):
```

```
                base_schema = self._merge_schema(base_schema, idms_schema)

            return base_schema

        # Last fallback: IDMS only.
        if self._has_schema_objects(idms_schema):
            return idms_schema

        raise ConversionError(
            "No usable schema source found. Provide canonical model JSON,
"
            "Phase 2 metadata JSON with records/relationships, DB2 DDL, "
            "or IDMS schema listing."
        )
    def _merge_ddl_into_canonical(
        self,
        canonical: SchemaModel,
        ddl_schema: SchemaModel,
    ) -> SchemaModel:
        return self._merge_schema(canonical, ddl_schema)

    def _merge_schema(
        self,
        base: SchemaModel,
        overlay: SchemaModel,
    ) -> SchemaModel:
        merged = deepcopy(base)

        for record_name, overlay_record in overlay.records.items():
            if record_name in merged.records:
                merged.records[record_name].primary_key = (
                    overlay_record.primary_key
                    or merged.records[record_name].primary_key
                )

                for field_name, overlay_field in
overlay_record.fields.items():
                    merged.records[record_name].fields[field_name] =
overlay_field
            else:
```

```
                merged.records[record_name] = overlay_record

        for set_name, overlay_rel in overlay.relationships.items():
            if set_name not in merged.relationships:
                merged.relationships[set_name] = overlay_rel
                continue

            rel = merged.relationships[set_name]
            rel.parent_record = overlay_rel.parent_record or
rel.parent_record
            rel.child_record = overlay_rel.child_record or
rel.child_record
            rel.parent_key = overlay_rel.parent_key or rel.parent_key
            rel.child_fk = overlay_rel.child_fk or rel.child_fk
            rel.cardinality = overlay_rel.cardinality or rel.cardinality

            if overlay_rel.order_by:
                rel.order_by = overlay_rel.order_by

        self._merge_dict(merged.field_map, overlay.field_map)
        self._merge_dict(merged.record_table_map,
overlay.record_table_map)
        self._merge_dict(merged.calc_key_map, overlay.calc_key_map)
        self._merge_dict(merged.set_ordering_map,
overlay.set_ordering_map)
        self._merge_dict(merged.navigation_intent,
overlay.navigation_intent)
        self._merge_dict(merged.nullable_fk_map, overlay.nullable_fk_map)
        self._merge_dict(merged.date_part_map, overlay.date_part_map)
        self._merge_dict(merged.output_semantics,
overlay.output_semantics)
        self._merge_dict(
            merged.paragraph_operation_graph,
            overlay.paragraph_operation_graph,
        )

        if hasattr(merged, "validation_messages") and hasattr(
            overlay,
            "validation_messages",
        ):
```

```
            merged.validation_messages.extend(overlay.validation_messages)

        if not getattr(merged, "schema_source", None):
            merged.schema_source = getattr(overlay, "schema_source", None)

        return merged

    def _merge_dict(self, target: dict, source: dict) -> None:
        for key, value in source.items():
            target[key] = value

    def _apply_relationship_overrides(
        self,
        schema: SchemaModel,
        relationship_overrides_json: str,
    ) -> SchemaModel:
        return RelationshipResolver().apply_overrides(
            schema=schema,
            relationship_overrides_json=relationship_overrides_json,
        )

    def _has_schema_objects(self, schema: SchemaModel) -> bool:
        if hasattr(schema, "has_schema_objects"):
            return schema.has_schema_objects()

        return bool(schema.records or schema.relationships)

    def _looks_like_idms_schema(self, text: str | None) -> bool:
        if not text:
            return False

        upper = text.upper()

        return (
            "RECORD NAME" in upper
            and "DBKEY POSITIONS" in upper
            and "DATA ITEM" in upper
        )

    def _debug_schema(self, schema: SchemaModel) -> None:
```

```
        print("SCHEMA SOURCE:", getattr(schema, "schema_source", None))
        print("RECORD COUNT:", len(schema.records))
        print("RELATIONSHIP COUNT:", len(schema.relationships))

        for record_name in ["DEPARTMENT", "EMPLOYEE", "EMPOSITION", "JOB",
"OFFICE"]:
            record = schema.records.get(record_name)

            if not record:
                print("MISSING RECORD:", record_name)
                continue

            print("RECORD:", record_name)

            for field_name, field in record.fields.items():
                print(
                    "  FIELD:",
                    field_name,
                    field.datatype,
                    field.length,
                    field.scale,
                    "NULLABLE=",
                    field.nullable,
                )

        print(
            "FIELD MAP HAS DEPT-NAME-0410:",
            "DEPT-NAME-0410" in schema.field_map,
        )
        print(
            "FIELD MAP HAS EMP-FIRST-NAME-0415:",
            "EMP-FIRST-NAME-0415" in schema.field_map,
        )
        print(
            "FIELD MAP HAS TITLE-0440:",
            "TITLE-0440" in schema.field_map,
        )
        print(
            "FIELD MAP HAS OFFICE-STREET-0450:",
            "OFFICE-STREET-0450" in schema.field_map,
```

```
        )

    def _merge_non_physical_metadata(
        self,
        physical_schema: SchemaModel,
        overlay: SchemaModel,
    ) -> SchemaModel:
        """
        Merge metadata from canonical into DDL physical schema without
overwriting
        DDL columns, DDL data types, DDL lengths, or DDL primary keys.
        """
        merged = deepcopy(physical_schema)

        for key, value in overlay.field_map.items():
            merged.field_map[key] = value

        for key, value in overlay.record_table_map.items():
            if key not in merged.record_table_map:
                merged.record_table_map[key] = value

        for key, value in overlay.calc_key_map.items():
            merged.calc_key_map[key] = value

        for key, value in overlay.set_ordering_map.items():
            merged.set_ordering_map[key] = value

        for key, value in overlay.navigation_intent.items():
            merged.navigation_intent[key] = value

        for key, value in overlay.nullable_fk_map.items():
            merged.nullable_fk_map[key] = value

        for key, value in overlay.date_part_map.items():
            merged.date_part_map[key] = value

        for key, value in overlay.output_semantics.items():
            merged.output_semantics[key] = value

        for key, value in overlay.paragraph_operation_graph.items():
```

```
            merged.paragraph_operation_graph[key] = value

        for set_name, relationship in overlay.relationships.items():
            if set_name not in merged.relationships:
                merged.relationships[set_name] = relationship

        if hasattr(merged, "validation_messages") and hasattr(
            overlay,
            "validation_messages",
        ):
            merged.validation_messages.extend(overlay.validation_messages)

        merged.schema_source = "DDL+CANONICAL_METADATA"

        return merged
```

src\idms_db2_converter\transformers\relationship_resolver.py

```
import json

from idms_db2_converter.exceptions import ConversionError
from idms_db2_converter.models import CobolAnalysis, Relationship,
SchemaModel


class RelationshipResolver:
    """
    Resolves COBOL IDMS set names to schema relationships.

    No business-specific set names are hard-coded.

    Resolution order:
    1. Use exact set name if already present.
    2. Use relationship overrides if provided.
    3. Use DDL-derived relationships if a COBOL set can be matched
unambiguously.
    """

    def apply_overrides(
        self,
        schema: SchemaModel,
        relationship_overrides_json: str,
    ) -> SchemaModel:
        try:
            payload = json.loads(relationship_overrides_json)
        except json.JSONDecodeError as exc:
            raise ConversionError(f"Invalid relationship override JSON:
{exc}") from exc

        self._apply_record_aliases(schema, payload)

        for item in payload.get("relationships", []):
            self._apply_relationship_override(schema, item)

        return schema
```

```
    def resolve_cobol_sets(
        self,
        schema: SchemaModel,
        analysis: CobolAnalysis,
    ) -> SchemaModel:
        references = self._collect_set_references(analysis)

        for set_name, child_records in references.items():
            if set_name in schema.relationships:
                continue

            matched = self._find_unambiguous_relationship(
                schema=schema,
                set_name=set_name,
                child_records=child_records,
            )

            if not matched:
                continue

            schema.relationships[set_name] = Relationship(
                set_name=set_name,
                parent_record=matched.parent_record,
                child_record=matched.child_record,
                cardinality=matched.cardinality,
                parent_key=matched.parent_key,
                child_fk=matched.child_fk,
                order_by=matched.order_by,
            )

            schema.add_validation_message(
                f"Set {set_name} resolved from existing schema
relationship "
                f"{matched.parent_record}-{matched.child_record}."
            )

        return schema

    def _collect_set_references(
        self,
```

```
        analysis: CobolAnalysis,
    ) -> dict[str, set[str]]:
        references: dict[str, set[str]] = {}

        for record_name, set_name in analysis.obtain_next:
            references.setdefault(set_name, set()).add(record_name)

        for set_name in analysis.obtain_owner_sets:
            references.setdefault(set_name, set())

        for set_name in analysis.find_first_sets:
            references.setdefault(set_name, set())

        return references

    def _apply_record_aliases(self, schema: SchemaModel, payload: dict) ->
None:
        for item in payload.get("record_aliases", []):
            record_name = item["record"].upper()
            table_name = item["table"].upper()

            if table_name not in schema.records:
                raise ConversionError(
                    f"Record alias {record_name} references unknown table
{table_name}."
                )

            schema.records[record_name] = schema.records[table_name]
            schema.record_table_map[record_name] = table_name

    def _apply_relationship_override(self, schema: SchemaModel, item:
dict) -> None:
        set_name = item["set_name"].upper()

        parent_record = self._upper_or_none(item.get("parent_record"))
        child_record = self._upper_or_none(item.get("child_record"))
        parent_key = self._upper_or_none(item.get("parent_key"))
        child_fk = self._upper_or_none(item.get("child_fk"))
        order_by = [value.upper() for value in item.get("order_by", [])]
```

```
        if set_name in schema.relationships:
            rel = schema.relationships[set_name]

            if parent_record:
                rel.parent_record = parent_record

            if child_record:
                rel.child_record = child_record

            if parent_key:
                rel.parent_key = parent_key

            if child_fk:
                rel.child_fk = child_fk

            if order_by:
                rel.order_by = order_by

            return

        if parent_record and child_record:
            self._create_explicit_relationship(
                schema=schema,
                set_name=set_name,
                parent_record=parent_record,
                child_record=child_record,
                parent_key=parent_key,
                child_fk=child_fk,
                order_by=order_by,
            )
            return

        matched = self._find_relationship_by_keys(
            schema=schema,
            parent_key=parent_key,
            child_fk=child_fk,
        )

        if matched:
            schema.relationships[set_name] = Relationship(
```

```
                set_name=set_name,
                parent_record=matched.parent_record,
                child_record=matched.child_record,
                cardinality=matched.cardinality,
                parent_key=parent_key or matched.parent_key,
                child_fk=child_fk or matched.child_fk,
                order_by=order_by or matched.order_by,
            )
            return

        raise ConversionError(
            f"Override references set {set_name}, but it could not be
resolved. "
            "Provide parent_record and child_record, or provide keys
matching a DDL foreign key."
        )

    def _create_explicit_relationship(
        self,
        schema: SchemaModel,
        set_name: str,
        parent_record: str,
        child_record: str,
        parent_key: str | None,
        child_fk: str | None,
        order_by: list[str],
    ) -> None:
        if parent_record not in schema.records:
            raise ConversionError(
                f"Set {set_name} parent_record {parent_record} is not
present in schema."
            )

        if child_record not in schema.records:
            raise ConversionError(
                f"Set {set_name} child_record {child_record} is not
present in schema."
            )

        parent = schema.records[parent_record]
```

```
        child = schema.records[child_record]

        resolved_parent_key = parent_key or parent.primary_key

        if not resolved_parent_key:
            raise ConversionError(
                f"Set {set_name} cannot resolve parent_key."
            )

        resolved_child_fk = child_fk

        if not resolved_child_fk and resolved_parent_key in child.fields:
            resolved_child_fk = resolved_parent_key

        if not resolved_child_fk:
            raise ConversionError(
                f"Set {set_name} cannot resolve child_fk."
            )

        schema.relationships[set_name] = Relationship(
            set_name=set_name,
            parent_record=parent_record,
            child_record=child_record,
            cardinality="1:N",
            parent_key=resolved_parent_key,
            child_fk=resolved_child_fk,
            order_by=order_by or [resolved_child_fk],
        )

    def _find_relationship_by_keys(
        self,
        schema: SchemaModel,
        parent_key: str | None,
        child_fk: str | None,
    ) -> Relationship | None:
        if not parent_key and not child_fk:
            return None

        matches = []
```

```
        for rel in schema.relationships.values():
            if parent_key and rel.parent_key != parent_key:
                continue

            if child_fk and rel.child_fk != child_fk:
                continue

            matches.append(rel)

        if len(matches) == 1:
            return matches[0]

        if len(matches) > 1:
            raise ConversionError(
                "Relationship override is ambiguous. Add parent_record and
child_record."
            )

        return None

    def _find_unambiguous_relationship(
        self,
        schema: SchemaModel,
        set_name: str,
        child_records: set[str],
    ) -> Relationship | None:
        candidates = list(schema.relationships.values())

        if child_records:
            filtered = [
                rel
                for rel in candidates
                if rel.child_record in child_records
            ]

            if filtered:
                candidates = filtered

        if len(candidates) == 1:
            return candidates[0]
```

```
        scored = []

        for rel in candidates:
            score = self._score_relationship(set_name, rel)

            if score > 0:
                scored.append((score, rel))

        if not scored:
            return None

        scored.sort(key=lambda item: item[0], reverse=True)

        top_score = scored[0][0]
        top_matches = [
            rel
            for score, rel in scored
            if score == top_score
        ]

        if len(top_matches) == 1:
            return top_matches[0]

        return None

    def _score_relationship(self, set_name: str, rel: Relationship) ->
int:
        tokens = [
            token
            for token in set_name.replace("_", "-").split("-")
            if token
        ]

        searchable_values = [
            rel.parent_record,
            rel.child_record,
            rel.parent_key or "",
            rel.child_fk or "",
        ]
```

```
        score = 0

        for token in tokens:
            for value in searchable_values:
                if self._token_matches(token, value):
                    score += 1

        return score

    def _token_matches(self, token: str, value: str) -> bool:
        token = token.upper()
        value = value.upper()

        return token in value or value in token

    def _upper_or_none(self, value: str | None) -> str | None:
        return value.upper() if value else None
```

src\idms_db2_converter\transformers\schema_merger.py

```
from copy import deepcopy
import re

from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.models import Relationship, SchemaModel


class SchemaMerger:
    """
    Merge DB2 DDL schema with IDMS schema listing.

    Core rule:
    - DDL wins for physical column definitions.
    - IDMS wins for legacy field names and field_map.
    """

    def merge_physical_with_idms(
        self,
```

```
        physical_schema: SchemaModel,
        idms_schema: SchemaModel,
    ) -> SchemaModel:
        merged = deepcopy(physical_schema)

        if not getattr(merged, "schema_source", None):
            merged.schema_source = "DDL"

        merged.schema_source = f"{merged.schema_source}+IDMS_SCHEMA"

        self._build_record_table_map(merged, idms_schema)
        self._build_field_map_from_idms_to_physical(merged, idms_schema)
        self._build_date_part_map_from_idms_to_physical(merged,
idms_schema)
        self._preserve_idms_set_names(merged, idms_schema)
        self._merge_validation_messages(merged, idms_schema)

        return merged

    def _build_record_table_map(
        self,
        merged: SchemaModel,
        idms_schema: SchemaModel,
    ) -> None:
        for idms_record_name in idms_schema.records:
            physical_record_name = self._find_matching_record(
                schema=merged,
                record_name=idms_record_name,
            )

            if physical_record_name:
                merged.record_table_map[idms_record_name] =
physical_record_name
            else:
                merged.record_table_map[idms_record_name] =
self._table_name(
                    idms_record_name
                )

    def _build_field_map_from_idms_to_physical(
```

```
        self,
        merged: SchemaModel,
        idms_schema: SchemaModel,
    ) -> None:
        for idms_record_name in idms_schema.records:
            physical_record_name =
merged.record_table_map.get(idms_record_name)

            if not physical_record_name:
                continue

            if physical_record_name not in merged.records:
                continue

            physical_record = merged.records[physical_record_name]

            for idms_field_name, meta in idms_schema.field_map.items():
                idms_host = str(meta.get("host", "")).upper()

                if not self._host_belongs_to_record(idms_host,
idms_record_name):
                    continue

                normalized_idms_field =
self._normalize_field_name(idms_field_name)

                physical_column = self._find_physical_column(
                    physical_record=physical_record,
                    normalized_idms_field=normalized_idms_field,
                    idms_host=idms_host,
                    idms_record_name=idms_record_name,
                )

                if not physical_column:
                    continue

                merged.field_map[idms_field_name.upper()] = {
                    "host": Naming.hv(idms_record_name, physical_column)
                }
```

```
    def _build_date_part_map_from_idms_to_physical(
        self,
        merged: SchemaModel,
        idms_schema: SchemaModel,
    ) -> None:
        for idms_field_name, meta in idms_schema.date_part_map.items():
            idms_host = str(meta.get("host", "")).upper()

            idms_record_name = self._find_record_for_host(
                host=idms_host,
                record_names=list(idms_schema.records.keys()),
            )

            if not idms_record_name:
                continue

            physical_record_name =
merged.record_table_map.get(idms_record_name)

            if not physical_record_name or physical_record_name not in
merged.records:
                continue

            physical_record = merged.records[physical_record_name]

            idms_column = self._column_from_host(idms_host,
idms_record_name)

            if not idms_column:
                continue

            physical_column = self._find_physical_column(
                physical_record=physical_record,
                normalized_idms_field=idms_column,
                idms_host=idms_host,
                idms_record_name=idms_record_name,
            )

            if not physical_column:
                continue
```

```
        column = physical_record.fields[physical_column]

        substring_start = meta.get("substring_start")
        substring_length = meta.get("substring_length")

        if column.datatype.upper() == "DATE":
            part = self._date_part_name(idms_field_name)

            if part == "YEAR":
                substring_start = 3
                substring_length = 2
            elif part == "MONTH":
                substring_start = 6
                substring_length = 2
            elif part == "DAY":
                substring_start = 9
                substring_length = 2

        merged.date_part_map[idms_field_name.upper()] = {
            "host": Naming.hv(idms_record_name, physical_column),
            "substring_start": substring_start,
            "substring_length": substring_length,
        }

def _preserve_idms_set_names(
    self,
    merged: SchemaModel,
    idms_schema: SchemaModel,
) -> None:
    for set_name, idms_rel in idms_schema.relationships.items():
        idms_parent = idms_rel.parent_record
        idms_child = idms_rel.child_record

        physical_parent = merged.record_table_map.get(idms_parent)
        physical_child = merged.record_table_map.get(idms_child)

        if not physical_parent or not physical_child:
            continue
```

```
            ddl_rel = self._find_relationship_by_records(
                schema=merged,
                parent_record=physical_parent,
                child_record=physical_child,
            )

            if ddl_rel:
                parent_key = ddl_rel.parent_key
                child_fk = ddl_rel.child_fk
                order_by = ddl_rel.order_by
            else:
                parent_key = self._find_matching_column_name(
                    record=merged.records[physical_parent],
                    name=idms_rel.parent_key or "",
                )

                child_fk = self._find_matching_column_name(
                    record=merged.records[physical_child],
                    name=idms_rel.child_fk or idms_rel.parent_key or "",
                )

                order_by = [child_fk] if child_fk else []

            if not parent_key or not child_fk:
                continue

            merged.relationships[set_name] = Relationship(
                set_name=set_name,
                parent_record=idms_parent,
                child_record=idms_child,
                cardinality=idms_rel.cardinality or "1:N",
                parent_key=parent_key,
                child_fk=child_fk,
                order_by=order_by or [child_fk],
            )

    def _find_physical_column(
        self,
        physical_record,
        normalized_idms_field: str,
```

```
        idms_host: str,
        idms_record_name: str,
    ) -> str | None:
        direct = self._find_matching_column_name(
            record=physical_record,
            name=normalized_idms_field,
        )

        if direct:
            return direct

        from_host = self._column_from_host(idms_host, idms_record_name)

        if from_host:
            direct = self._find_matching_column_name(
                record=physical_record,
                name=from_host,
            )

            if direct:
                return direct

        singular = self._singularize(normalized_idms_field)

        if singular != normalized_idms_field:
            direct = self._find_matching_column_name(
                record=physical_record,
                name=singular,
            )

            if direct:
                return direct

        return self._find_flattened_occurs_column(
            physical_record=physical_record,
            name=normalized_idms_field,
        )

    def _find_flattened_occurs_column(
        self,
```

```
        physical_record,
        name: str,
    ) -> str | None:
        candidates = [
            column_name
            for column_name in physical_record.fields
            if column_name.upper().startswith(name.upper() + "_")
        ]

        if not candidates:
            singular = self._singularize(name)
            candidates = [
                column_name
                for column_name in physical_record.fields
                if column_name.upper().startswith(singular.upper() + "_")
            ]

        if not candidates:
            return None

        return sorted(candidates)[0]

    def _find_matching_record(
        self,
        schema: SchemaModel,
        record_name: str,
    ) -> str | None:
        record_name = record_name.upper()

        if record_name in schema.records:
            return record_name

        normalized = self._normalize_record_name(record_name)

        for candidate in schema.records:
            if self._normalize_record_name(candidate) == normalized:
                return candidate

        return None
```

```
    def _find_matching_column_name(
        self,
        record,
        name: str,
    ) -> str | None:
        normalized = self._normalize_field_name(name)

        if normalized in record.fields:
            return normalized

        for column_name in record.fields:
            if column_name.upper() == normalized:
                return column_name

        return None

    def _find_relationship_by_records(
        self,
        schema: SchemaModel,
        parent_record: str,
        child_record: str,
    ) -> Relationship | None:
        normalized_parent = self._normalize_record_name(parent_record)
        normalized_child = self._normalize_record_name(child_record)

        for relationship in schema.relationships.values():
            if (
                self._normalize_record_name(relationship.parent_record)
                == normalized_parent
                and self._normalize_record_name(relationship.child_record)
                == normalized_child
            ):
                return relationship

        return None

    def _host_belongs_to_record(
        self,
        host: str,
        record_name: str,
```

```
    ) -> bool:
        normalized_host = self._normalize_token(host)
        normalized_record = self._normalize_token(record_name)

        return normalized_host.startswith(f"HV_{normalized_record}_")

    def _find_record_for_host(
        self,
        host: str,
        record_names: list[str],
    ) -> str | None:
        normalized_host = self._normalize_token(host)

        for record_name in sorted(record_names, key=len, reverse=True):
            normalized_record = self._normalize_token(record_name)

            if normalized_host.startswith(f"HV_{normalized_record}_"):
                return record_name

        return None

    def _normalize_token(
        self,
        value: str,
    ) -> str:
        return value.upper().replace("-", "_")

    def _column_from_host(
        self,
        host: str,
        record_name: str,
    ) -> str | None:
        normalized_host = self._normalize_token(host)
        normalized_record = self._normalize_token(record_name)

        prefix = f"HV_{normalized_record}_"

        if not normalized_host.startswith(prefix):
            return None
```

```
        return normalized_host[len(prefix):].upper()

    def _date_part_name(
        self,
        idms_field_name: str,
    ) -> str | None:
        normalized = self._normalize_field_name(idms_field_name)

        if normalized.endswith("_YEAR"):
            return "YEAR"

        if normalized.endswith("_MONTH"):
            return "MONTH"

        if normalized.endswith("_DAY"):
            return "DAY"

        return None

    def _normalize_record_name(
        self,
        value: str,
    ) -> str:
        return value.upper().replace("-", "_")

    def _normalize_field_name(
        self,
        value: str,
    ) -> str:
        value = value.upper()
        value = re.sub(r"-\d{4}$", "", value)
        return value.replace("-", "_")

    def _singularize(
        self,
        value: str,
    ) -> str:
        value = value.upper()

        if value.endswith("IES"):
```

```
            return value[:-3] + "Y"

        if value.endswith("S"):
            return value[:-1]

        return value

    def _table_name(
        self,
        record_name: str,
    ) -> str:
        return record_name.upper().replace("-", "_")

    def _merge_validation_messages(
        self,
        merged: SchemaModel,
        overlay: SchemaModel,
    ) -> None:
        if hasattr(merged, "validation_messages") and hasattr(
            overlay,
            "validation_messages",
        ):
            merged.validation_messages.extend(overlay.validation_messages)
```

src\idms_db2_converter\transformers\cobol_cleanup.py

```
import re


class CobolCleanup:
    def clean(self, text: str) -> str:
        text = self._remove_duplicate_fetch_paragraphs(text)
        text = self._remove_stray_duplicate_fetch_tail(text)
        text = self._normalize_sqlcode_then(text)
        text = self._remove_end_processing_sqlcode_guard(text)
        text = self._normalize_exec_sql_indentation(text)
        text = self._normalize_close_file_indentation(text)
        text = self._fix_end_if_same_line_statement(text)
```

```
        text = self._ensure_final_end_if_periods(text)
        text = self._remove_duplicate_blank_lines(text)
        return text

    def _remove_duplicate_fetch_paragraphs(self, text: str) -> str:
        pattern = re.compile(

r"(?P<para>^\s*(?P<name>FETCH-[A-Z0-9-]+)\.\n.*?)(?=^\s*[A-Z0-9-]+\.\s*$)"
,
            re.IGNORECASE | re.MULTILINE | re.DOTALL,
        )

        seen = set()
        output = []
        last_end = 0

        for match in pattern.finditer(text):
            name = match.group("name").upper()
            output.append(text[last_end:match.start()])

            if name not in seen:
                seen.add(name)
                output.append(match.group("para"))

            last_end = match.end()

        output.append(text[last_end:])
        return "".join(output)

    def _remove_stray_duplicate_fetch_tail(self, text: str) -> str:
        return re.sub(
            r"\n\s*END-EXEC\.\s*"
            r"\n\s*IF SQLCODE NOT = 0 AND SQLCODE NOT = 100\s*"
            r"\n\s*PERFORM SQL-ERROR\s*"
            r"\n\s*END-IF\.\s*"
            r"(?=\n\s*MAIN-LINE\.)",
            "\n",
            text,
            flags=re.IGNORECASE,
        )
```

```
    def _normalize_sqlcode_then(self, text: str) -> str:
        return re.sub(
            r"IF SQLCODE = 100 THEN",
            "IF SQLCODE = 100",
            text,
            flags=re.IGNORECASE,
        )

    def _remove_end_processing_sqlcode_guard(self, text: str) -> str:
        return re.sub(
            r"(\n\s*END-PROCESSING\.\n)"
            r"\s*IF SQLCODE NOT = 0 AND SQLCODE NOT = 100\s*"
            r"\n\s*PERFORM SQL-ERROR\s*"
            r"\n\s*END-IF\.\s*",
            r"\1",
            text,
            flags=re.IGNORECASE,
        )

    def _normalize_exec_sql_indentation(self, text: str) -> str:
        return re.sub(
            r"\n\s{8,}EXEC SQL",
            "\n       EXEC SQL",
            text,
            flags=re.IGNORECASE,
        )

    def _normalize_close_file_indentation(self, text: str) -> str:
        return re.sub(
            r"\n\s*CLOSE\s+([A-Z0-9-]+)\.",
            r"\n       CLOSE \1.",
            text,
            flags=re.IGNORECASE,
        )

    def _fix_end_if_same_line_statement(self, text: str) -> str:
        text = re.sub(
            r"END-IF\.\s+(MOVE\s+)",
            "END-IF.\n       MOVE ",
```

```
            text,
            flags=re.IGNORECASE,
        )

        text = re.sub(
            r"END-IF\.\s+(PERFORM\s+)",
            "END-IF.\n       PERFORM ",
            text,
            flags=re.IGNORECASE,
        )

        return text

    def _ensure_final_end_if_periods(self, text: str) -> str:
        text = re.sub(
            r"\n(\s*)END-IF\s*\n(\s*[A-Z0-9-]+\.)",
            r"\n\1END-IF.\n\2",
            text,
            flags=re.IGNORECASE,
        )

        text = re.sub(
            r"\n(\s*)END-IF\s*\n(\s*READ\s+)",
            r"\n\1END-IF.\n\2",
            text,
            flags=re.IGNORECASE,
        )

        text = re.sub(
            r"\n(\s*)END-IF\s*\n(\s*MOVE\s+)",
            r"\n\1END-IF.\n\2",
            text,
            flags=re.IGNORECASE,
        )

        return text


    def _remove_duplicate_blank_lines(self, text: str) -> str:
        return re.sub(r"\n{4,}", "\n\n\n", text)
```

src\idms_db2_converter\transformers\field_rewriter.py

```python
import re

from idms_db2_converter.models import SchemaModel


class FieldRewriter:
    def __init__(self, schema: SchemaModel):
        self.schema = schema

    def rewrite(self, text: str) -> str:
        before_procedure, procedure = self._split_procedure_division(text)

        if not procedure:
            return text

        procedure = self._rewrite_date_part_moves(procedure)
        procedure = self._rewrite_field_references(procedure)

        return before_procedure + procedure

    def _split_procedure_division(self, text: str) -> tuple[str, str]:
        match = re.search(
            r"^\s*PROCEDURE\s+DIVISION\.",
            text,
            flags=re.IGNORECASE | re.MULTILINE,
        )

        if not match:
            return text, ""

        return text[:match.start()], text[match.start():]

    def _rewrite_date_part_moves(self, text: str) -> str:
        return self._rewrite_lines(
            text=text,
            replacer=self._rewrite_date_part_line,
        )
```

```
    def _rewrite_field_references(self, text: str) -> str:
        return self._rewrite_lines(
            text=text,
            replacer=self._rewrite_field_line,
        )

    def _rewrite_lines(self, text: str, replacer) -> str:
        lines = []
        in_exec_sql = False

        for line in text.splitlines():
            upper = line.upper()

            if "EXEC SQL" in upper:
                in_exec_sql = True
                lines.append(line)
                continue

            if in_exec_sql:
                lines.append(line)

                if "END-EXEC" in upper:
                    in_exec_sql = False

                continue

            if line.strip().startswith("*"):
                lines.append(line)
                continue

            lines.append(replacer(line))

        return "\n".join(lines)

    def _rewrite_date_part_line(self, line: str) -> str:
        for idms_field, meta in self.schema.date_part_map.items():
            host = meta.get("host")
            start = meta.get("substring_start")
            length = meta.get("substring_length")
```

```
            if not host or not start or not length:
                continue

            pattern = re.compile(

rf"(?<![A-Z0-9-])MOVE\s+{re.escape(idms_field)}(?![A-Z0-9-])\s+TO\s+([A-Z0
-9-]+)\.?",
                re.IGNORECASE,
            )

            replacement = rf"MOVE {host}({start}:{length}) TO \1."

            line = self._replace_outside_literals(line, pattern,
replacement)

        return line

    def _rewrite_field_line(self, line: str) -> str:
        field_names = sorted(
            self.schema.field_map.keys(),
            key=len,
            reverse=True,
        )

        for idms_field in field_names:
            if idms_field.upper() in self.schema.date_part_map:
                continue

            meta = self.schema.field_map[idms_field]
            host = meta.get("host")

            if not host:
                continue

            pattern = re.compile(
                rf"(?<![A-Z0-9-]){re.escape(idms_field)}(?![A-Z0-9-])",
                re.IGNORECASE,
            )

            line = self._replace_outside_literals(line, pattern, host)
```

```
        return line

    def _replace_outside_literals(
        self,
        line: str,
        pattern: re.Pattern,
        replacement: str,
    ) -> str:
        parts = re.split(r"('(?:[^']|'')*')", line)

        for index, part in enumerate(parts):
            if index % 2 == 1:
                continue

            parts[index] = pattern.sub(replacement, part)

        return "".join(parts)
```

src\idms_db2_converter\transformers\metadata_helpers.py

```
from idms_db2_converter.generators.cobol_builder import Move
from idms_db2_converter.models import SchemaModel


class MetadataHelpers:
    def __init__(self, schema: SchemaModel):
        self.schema = schema

    def lookup_by_name(self, name: str) -> dict | None:
        for item in self.schema.output_semantics.get("lookups", []):
            if item.get("name", "").upper() == name.upper():
                return item

        return None

    def lookup_by_owner_set(self, owner_set: str) -> dict | None:
        for item in self.schema.output_semantics.get("lookups", []):
            if item.get("owner_set", "").upper() == owner_set.upper():
                return item

        return None

    def lookup_by_first_member_set(self, first_member_set: str) -> dict |
None:
        for item in self.schema.output_semantics.get("lookups", []):
            if item.get("first_member_set", "").upper() ==
first_member_set.upper():
                return item

        return None

    def moves_to_nodes(self, moves: list[dict]) -> list[Move]:
        nodes = []

        for move in moves:
            source = self.map_source(move["from"])
            target = move["to"]
            nodes.append(Move(source=source, target=target))
```

```
        return nodes

    def map_source(self, source: str) -> str:
        source_upper = source.upper()

        if source_upper in self.schema.date_part_map:
            meta = self.schema.date_part_map[source_upper]
            return
f"{meta['host']}({meta['substring_start']}:{meta['substring_length']})"

        if source_upper in self.schema.field_map:
            return self.schema.field_map[source_upper].get("host", source)

        return source
```

src\idms_db2_converter\transformers\operation_graph_transformer.py

```python
import re

from idms_db2_converter.generators.cobol_builder import (
    Continue,
    ExecSql,
    IfBlock,
    Move,
    Perform,
    PerformUntil,
    PreRenderedBlock,
    ReadAtEndMove,
    Write,
    render_nodes,
    render_paragraph,
)
from idms_db2_converter.generators.fetch_paragraphs import
FetchParagraphGenerator
from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.generators.sql_snippets import SqlSnippets
from idms_db2_converter.models import SchemaModel
from idms_db2_converter.transformers.metadata_helpers import
MetadataHelpers


class OperationGraphTransformer:
    def __init__(self, schema: SchemaModel):
        self.schema = schema
        self.sql = SqlSnippets(schema)
        self.meta = MetadataHelpers(schema)
        self.fetch_generator = FetchParagraphGenerator(schema)
        self.used_cursor_sets: set[str] = set()

    def has_graph(self) -> bool:
        return bool(self.schema.paragraph_operation_graph)

    def transform(self, text: str) -> str:
        for paragraph_name, operations in
self.schema.paragraph_operation_graph.items():
```

```
            rendered = self._render_paragraph(paragraph_name, operations)
            text = self._replace_paragraph(text, paragraph_name, rendered)

        return text

    def _replace_paragraph(self, text: str, paragraph_name: str,
replacement: str) -> str:
        pattern = re.compile(

rf"^\s*{re.escape(paragraph_name)}\.\s*\n.*?(?=^\s*[A-Z0-9-]+\.\s*$)",
            re.IGNORECASE | re.MULTILINE | re.DOTALL,
        )

        if pattern.search(text):
            return pattern.sub(replacement + "\n", text, count=1)

        return text + "\n" + replacement + "\n"

    def _render_paragraph(self, paragraph_name: str, operations:
list[dict]) -> str:
        nodes = self._render_operations(operations)
        return render_paragraph(paragraph_name, nodes)

    def _render_operations(self, operations: list[dict]) -> list:
        nodes = []

        for operation in operations:
            op_type = operation.get("type", "").upper()

            if op_type == "MOVE":
                nodes.append(
                    Move(
                        source=self.meta.map_source(operation["from"]),
                        target=operation["to"],
                    )
                )

            elif op_type == "MOVE_DATE_PART":
                nodes.append(
                    Move(
```

```
                    source=self.meta.map_source(operation["from"]),
                    target=operation["to"],
                )
            )

        elif op_type == "PERFORM":
            nodes.append(Perform(paragraph=operation["paragraph"]))

        elif op_type == "WRITE":
            nodes.append(
                Write(
                    record=operation["record"],
                    source=operation.get("source"),
                )
            )

        elif op_type == "READ_NEXT_INPUT":
            at_end_move = operation.get("at_end_move", {})
            nodes.append(
                ReadAtEndMove(
                    file_name=operation["file"],
                    source=at_end_move.get("from", "Y"),
                    target=at_end_move.get("to", "EOF-SW"),
                )
            )

        elif op_type == "CONTINUE":
            nodes.append(Continue())

        elif op_type == "OBTAIN_CALC":
            nodes.append(self._node_obtain_calc(operation))

        elif op_type == "CURSOR_LOOP":
            nodes.append(self._node_cursor_loop(operation))

        elif op_type == "FIRST_MEMBER_TO_OWNER_LOOKUP":

nodes.extend(self._nodes_first_member_to_owner_lookup(operation))

        elif op_type == "OWNER_LOOKUP":
```

```
                nodes.extend(self._nodes_owner_lookup(operation))

        return nodes

    def _node_obtain_calc(self, operation: dict):
        record = operation["record"].upper()
        select_sql = self.sql.select_for_record_by_pk(record)

        not_found_nodes =
self._render_operations(operation.get("not_found", []))
        success_nodes = self._render_operations(operation.get("success",
[]))

        lines = []
        lines.extend(ExecSql(select_sql).render(indent=7))
        lines.append("")
        lines.extend(
            IfBlock(
                condition="SQLCODE = 100",
                then_body=not_found_nodes,
                else_body=[
                    IfBlock(
                        condition="SQLCODE NOT = 0",
                        then_body=[Perform("SQL-ERROR")],
                        else_body=success_nodes,
                    )
                ],
            ).render(indent=7, final=True)
        )

        return PreRenderedBlock(lines)

    def _node_cursor_loop(self, operation: dict):
        set_name = operation["set"].upper()
        self.used_cursor_sets.add(set_name)

        cursor_name = Naming.cursor(set_name)
        fetch_paragraph = self.fetch_generator.paragraph_name(set_name)

        process_paragraph = operation["process_paragraph"]
```

```
        exit_paragraph = operation.get("exit_paragraph")
        empty_nodes = self._render_operations(operation.get("empty", []))
        before_loop_nodes =
self._render_operations(operation.get("before_loop", []))

        perform_target = process_paragraph
        if exit_paragraph:
            perform_target = f"{process_paragraph} THRU {exit_paragraph}"

        cursor_open = ExecSql(
            "\n".join(
                [
                    "EXEC SQL",
                    f"     OPEN {cursor_name}",
                    "END-EXEC.",
                ]
            )
        )

        cursor_close = ExecSql(
            "\n".join(
                [
                    "EXEC SQL",
                    f"     CLOSE {cursor_name}",
                    "END-EXEC.",
                ]
            )
        )

        loop_nodes = before_loop_nodes + [
            PerformUntil(
                condition="SQLCODE = 100",
                body=[
                    Perform(perform_target),
                    Perform(fetch_paragraph),
                ],
            )
        ]

        return IfBlock(
```

```
            condition="SQLCODE NOT = 0",
            then_body=[Perform("SQL-ERROR")],
            else_body=[
                Perform(fetch_paragraph),
                IfBlock(
                    condition="SQLCODE = 100",
                    then_body=empty_nodes,
                    else_body=loop_nodes,
                ),
                cursor_close,
            ],
        ).with_prefix(cursor_open)

    def _nodes_first_member_to_owner_lookup(self, operation: dict) ->
list:
        lookup_name = operation.get("name")
        first_member_set = operation["first_member_set"].upper()
        owner_set = operation["owner_set"].upper()

        lookup = self.meta.lookup_by_name(lookup_name) if lookup_name else
None
        if not lookup:
            lookup =
self.meta.lookup_by_first_member_set(first_member_set)

        not_found_moves = lookup.get("not_found_moves", []) if lookup else
[]
        success_moves = lookup.get("success_moves", []) if lookup else []

        first_select =
ExecSql(self.sql.select_first_child_for_set(first_member_set))
        owner_select = ExecSql(self.sql.select_for_owner(owner_set))

        not_found_nodes = self.meta.moves_to_nodes(not_found_moves)
        owner_not_found_nodes = self.meta.moves_to_nodes(not_found_moves)
        success_nodes = self.meta.moves_to_nodes(success_moves) or
[Continue()]

        return [
            first_select,
```

```
            IfBlock(
                condition="SQLCODE = 100",
                then_body=not_found_nodes,
                else_body=[
                    IfBlock(
                        condition="SQLCODE NOT = 0",
                        then_body=[Perform("SQL-ERROR")],
                        else_body=[
                            owner_select,
                            IfBlock(
                                condition="SQLCODE = 100",
                                then_body=owner_not_found_nodes,
                                else_body=[
                                    IfBlock(
                                        condition="SQLCODE NOT = 0",
                                        then_body=[Perform("SQL-ERROR")],
                                        else_body=success_nodes,
                                    )
                                ],
                            ),
                        ],
                    )
                ],
            ),
        ]

    def _nodes_owner_lookup(self, operation: dict) -> list:
        lookup_name = operation.get("name")
        owner_set = operation["owner_set"].upper()

        lookup = self.meta.lookup_by_name(lookup_name) if lookup_name else
None
        if not lookup:
            lookup = self.meta.lookup_by_owner_set(owner_set)

        not_found_moves = lookup.get("not_found_moves", []) if lookup else
[]
        success_moves = lookup.get("success_moves", []) if lookup else []

        not_found_nodes = self.meta.moves_to_nodes(not_found_moves)
```

```
        success_nodes = self.meta.moves_to_nodes(success_moves) or
[Continue()]
        owner_select = ExecSql(self.sql.select_for_owner(owner_set))

        nullable_meta = self.schema.nullable_fk_map.get(owner_set)
        null_indicator = nullable_meta.get("null_indicator") if
nullable_meta else None

        if null_indicator:
            return [
                IfBlock(
                    condition=f"{null_indicator} < 0",
                    then_body=not_found_nodes,
                    else_body=[
                        owner_select,
                        IfBlock(
                            condition="SQLCODE = 0",
                            then_body=success_nodes,
                            else_body=[
                                IfBlock(
                                    condition="SQLCODE = 100",
                                    then_body=not_found_nodes,
                                    else_body=[Perform("SQL-ERROR")],
                                )
                            ],
                        ),
                    ],
                )
            ]

        return [
            owner_select,
            IfBlock(
                condition="SQLCODE = 0",
                then_body=success_nodes,
                else_body=[
                    IfBlock(
                        condition="SQLCODE = 100",
                        then_body=not_found_nodes,
                        else_body=[Perform("SQL-ERROR")],
```

```
                    )
                ],
            ),
        ]
```

src\idms_db2_converter\transformers\paragraph_rewriter.py

```
import re

from idms_db2_converter.generators.fetch_paragraphs import
FetchParagraphGenerator
from idms_db2_converter.generators.naming import Naming
from idms_db2_converter.generators.sql_snippets import SqlSnippets
from idms_db2_converter.models import SchemaModel
from idms_db2_converter.transformers.structured_blocks import
StructuredBlocks


class ParagraphRewriter:
    def __init__(self, schema: SchemaModel):
        self.schema = schema
        self.sql = SqlSnippets(schema)
        self.fetch_generator = FetchParagraphGenerator(schema)
        self.controlled_loop_sets: set[str] = set()

    def rewrite(self, text: str) -> str:
        text = self._rewrite_cursor_loop_blocks(text)
        text = self._rewrite_find_first_owner_chain_blocks(text)
        text = self._rewrite_empty_else_obtain_owner_blocks(text)
        text = self._rewrite_empty_else_find_first_blocks(text)
        text = self._rewrite_obtain_calc_if_blocks(text)
        text = self._rewrite_remaining_obtain_calc(text)
        text = self._rewrite_remaining_obtain_next(text)
        text = self._rewrite_remaining_find_first(text)
        text = self._rewrite_remaining_obtain_owner(text)
        text = self._rewrite_status_tokens(text)
        return text

    def _rewrite_cursor_loop_blocks(self, text: str) -> str:
        for set_name in self.schema.relationships:
            text = self._rewrite_cursor_loop_for_set(text, set_name)

        return text
```

```
    def _rewrite_cursor_loop_for_set(self, text: str, set_name: str) ->
str:
        pattern = re.compile(
            rf"""
            IF\s+{re.escape(set_name)}\s+IS\s+NOT\s+EMPTY\s+THEN
            (?P<before_loop>.*?)
            PERFORM\s+(?P<walk>[A-Z0-9-]+)\s+THRU\s+(?P<exit>[A-Z0-9-]+)
            \s+UNTIL\s+DB-END-OF-SET
            \s+ELSE
            (?P<empty_body>.*?)
            (?=\.\s*READ|\.\s*[A-Z0-9-]+\.)
            """,
            re.IGNORECASE | re.DOTALL | re.VERBOSE,
        )

        def replace(match: re.Match) -> str:
            self.controlled_loop_sets.add(set_name.upper())

            return StructuredBlocks.cursor_loop_block(
                cursor_name=Naming.cursor(set_name),

fetch_paragraph=self.fetch_generator.paragraph_name(set_name),
                empty_body=self._clean_block(match.group("empty_body")),

before_loop_body=self._clean_block(match.group("before_loop")),
                walk_paragraph=match.group("walk").upper(),
                walk_exit=match.group("exit").upper(),
                indent=7,
            )

        return pattern.sub(replace, text)

    def _rewrite_find_first_owner_chain_blocks(self, text: str) -> str:
        for first_set in self.schema.relationships:
            for owner_set in self.schema.relationships:
                text = self._rewrite_one_find_first_owner_chain(
                    text=text,
                    first_set=first_set,
                    owner_set=owner_set,
                )
```

```
        return text

    def _rewrite_one_find_first_owner_chain(
        self,
        text: str,
        first_set: str,
        owner_set: str,
    ) -> str:
        pattern = re.compile(
            rf"""
            IF\s+{re.escape(first_set)}\s+IS\s+EMPTY
            (?P<first_empty>.*?)
            ELSE\s+
            FIND\s+FIRST\s+WITHIN\s+{re.escape(first_set)}\.?
            (?P<after_find>.*?)
            IF\s+NOT\s+{re.escape(owner_set)}\s+MEMBER
            (?P<owner_missing>.*?)
            ELSE\s+
            OBTAIN\s+OWNER\s+WITHIN\s+{re.escape(owner_set)}\.?
            (?P<success>.*?)
            (?=
                \n\s*IF\s+[A-Z0-9-]+\s+IS\s+EMPTY
                |
                \n\s*MOVE\s+EMP-DETAIL-LINE
                |
                \n\s*PERFORM\s+
                |
                \n\s*[A-Z0-9-]+-EXIT\.
            )
            """,
            re.IGNORECASE | re.DOTALL | re.VERBOSE,
        )

        def replace(match: re.Match) -> str:
            first_empty = self._remove_idms_status_lines(
                self._clean_block(match.group("first_empty"))
            )
            owner_missing = self._remove_idms_status_lines(
                self._clean_block(match.group("owner_missing"))
```

```
            )
            success = self._remove_idms_status_lines(
                self._clean_block(match.group("success"))
            )

            if not success:
                success = "CONTINUE"

            return StructuredBlocks.select_first_then_owner_chain(

first_select_sql=self.sql.select_first_child_for_set(first_set),
                owner_select_sql=self.sql.select_for_owner(owner_set),
                first_not_found_body=first_empty,
                owner_not_found_body=owner_missing,
                success_body=success,
                indent=7,
            )

        return pattern.sub(replace, text)

    def _rewrite_empty_else_obtain_owner_blocks(self, text: str) -> str:
        for set_name in self.schema.relationships:
            pattern = re.compile(
                rf"""
                IF\s+{re.escape(set_name)}\s+IS\s+EMPTY
                (?P<empty_body>.*?)
                ELSE\s+
                OBTAIN\s+OWNER\s+WITHIN\s+{re.escape(set_name)}\.?
                (?P<success_body>.*?)
                (?=
                    \n\s*MOVE\s+EMP-DETAIL-LINE
                    |
                    \n\s*PERFORM\s+
                    |
                    \n\s*[A-Z0-9-]+-EXIT\.
                )
                """,
                re.IGNORECASE | re.DOTALL | re.VERBOSE,
            )
```

```
            def replace(match: re.Match, set_name=set_name) -> str:
                empty_body = self._remove_idms_status_lines(
                    self._clean_block(match.group("empty_body"))
                )
                success_body = self._remove_idms_status_lines(
                    self._clean_block(match.group("success_body"))
                )

                if not success_body:
                    success_body = "CONTINUE"

                indicator =
self.sql.nullable_fk_indicator_for_set(set_name)

                if indicator:
                    return StructuredBlocks.nullable_owner_block(
                        null_indicator=indicator,
                        null_body=empty_body,

owner_select_sql=self.sql.select_for_owner(set_name),
                        success_body=success_body,
                        fallback_body=empty_body,
                        indent=7,
                    )

                return StructuredBlocks.owner_select_block(
                    owner_select_sql=self.sql.select_for_owner(set_name),
                    success_body=success_body,
                    fallback_body=empty_body,
                    indent=7,
                )

            text = pattern.sub(replace, text)

        return text

    def _rewrite_empty_else_find_first_blocks(self, text: str) -> str:
        for set_name in self.schema.relationships:
            pattern = re.compile(
                rf"""
```

```
            IF\s+{re.escape(set_name)}\s+IS\s+EMPTY
            (?P<empty_body>.*?)
            ELSE\s+
            FIND\s+FIRST\s+WITHIN\s+{re.escape(set_name)}\.?
            (?P<success_body>.*?)
            (?=
                \n\s*IF\s+[A-Z0-9-]+\s+IS\s+EMPTY
                |
                \n\s*MOVE\s+EMP-DETAIL-LINE
                |
                \n\s*PERFORM\s+
                |
                \n\s*[A-Z0-9-]+-EXIT\.
            )
            """,
            re.IGNORECASE | re.DOTALL | re.VERBOSE,
        )

        def replace(match: re.Match, set_name=set_name) -> str:
            empty_body = self._remove_idms_status_lines(
                self._clean_block(match.group("empty_body"))
            )
            success_body = self._remove_idms_status_lines(
                self._clean_block(match.group("success_body"))
            )

            if not success_body:
                success_body = "CONTINUE"

            return StructuredBlocks.sqlcode_select_block(
                sql=self.sql.select_first_child_for_set(set_name),
                not_found_body=empty_body,
                success_body=success_body,
                indent=7,
            )

        text = pattern.sub(replace, text)

    return text
```

```
    def _rewrite_obtain_calc_if_blocks(self, text: str) -> str:
        pattern = re.compile(
            r"""
            OBTAIN\s+CALC\s+(?P<record>[A-Z0-9-]+)\.?
            \s+
            IF\s+DB-REC-NOT-FOUND\s+THEN
            (?P<not_found>.*?)
            ELSE
            (?P<success>.*?)
            (?=\n\s*READ\s+)
            """,
            re.IGNORECASE | re.DOTALL | re.VERBOSE,
        )

        def replace(match: re.Match) -> str:
            record = match.group("record").upper()
            not_found = self._remove_idms_status_lines(
                self._clean_block(match.group("not_found"))
            )
            success = self._remove_idms_status_lines(
                self._clean_block(match.group("success"))
            )

            return StructuredBlocks.sqlcode_select_block(
                sql=self.sql.select_for_record_by_pk(record),
                not_found_body=not_found,
                success_body=success,
                indent=7,
            )

        return pattern.sub(replace, text)

    def _rewrite_remaining_obtain_calc(self, text: str) -> str:
        pattern = re.compile(
            r"^\s*OBTAIN\s+CALC\s+([A-Z0-9-]+)\.?\s*$",
            re.IGNORECASE | re.MULTILINE,
        )

        return pattern.sub(
```

```
            lambda m:
self.sql.select_for_record_by_pk(m.group(1).upper()),
            text,
        )

    def _rewrite_remaining_obtain_next(self, text: str) -> str:
        pattern = re.compile(

r"^\s*OBTAIN\s+NEXT\s+([A-Z0-9-]+)\s+WITHIN\s+([A-Z0-9-]+)\.?\s*$",
            re.IGNORECASE | re.MULTILINE,
        )

        def replace(match: re.Match) -> str:
            set_name = match.group(2).upper()

            if set_name in self.controlled_loop_sets:
                return "       CONTINUE."

            return f"       PERFORM
{self.fetch_generator.paragraph_name(set_name)}"

        return pattern.sub(replace, text)

    def _rewrite_remaining_find_first(self, text: str) -> str:
        pattern = re.compile(
            r"^\s*FIND\s+FIRST\s+WITHIN\s+([A-Z0-9-]+)\.?\s*$",
            re.IGNORECASE | re.MULTILINE,
        )

        return pattern.sub(
            lambda m:
self.sql.select_first_child_for_set(m.group(1).upper()),
            text,
        )

    def _rewrite_remaining_obtain_owner(self, text: str) -> str:
        pattern = re.compile(
            r"^\s*OBTAIN\s+OWNER\s+WITHIN\s+([A-Z0-9-]+)\.?\s*$",
            re.IGNORECASE | re.MULTILINE,
        )
```

```
        return pattern.sub(
            lambda m: self.sql.select_for_owner(m.group(1).upper()),
            text,
        )

    def _rewrite_status_tokens(self, text: str) -> str:
        text = re.sub(
            r"\bDB-REC-NOT-FOUND\b",
            "SQLCODE = 100",
            text,
            flags=re.IGNORECASE,
        )

        text = re.sub(
            r"\bDB-END-OF-SET\b",
            "SQLCODE = 100",
            text,
            flags=re.IGNORECASE,
        )

        text = re.sub(
            r"IF\s+NOT\s+[A-Z0-9-]+\s+MEMBER",
            "IF SQLCODE = 100",
            text,
            flags=re.IGNORECASE,
        )

        return text

    def _remove_idms_status_lines(self, text: str) -> str:
        text = re.sub(
            r"^\s*PERFORM\s+IDMS-STATUS\.?\s*$",
            "",
            text,
            flags=re.IGNORECASE | re.MULTILINE,
        )

        text = re.sub(
            r"^\s*IF SQLCODE NOT = 0 AND SQLCODE NOT = 100\s*"
```

```
            r"\n\s*PERFORM SQL-ERROR\s*"
            r"\n\s*END-IF\.?\s*$",
            "",
            text,
            flags=re.IGNORECASE | re.MULTILINE,
        )

        return text.strip()

    def _clean_block(self, value: str) -> str:
        value = value.strip()

        if value.endswith("."):
            value = value[:-1].rstrip()

        return value
```

src\idms_db2_converter\transformers\retrieval_transformer.py

```
import re

from idms_db2_converter.generators.cursors import CursorGenerator
from idms_db2_converter.generators.fetch_paragraphs import
FetchParagraphGenerator
from idms_db2_converter.generators.host_variables import
HostVariableGenerator
from idms_db2_converter.models import CobolAnalysis, SchemaModel
from idms_db2_converter.transformers.cobol_cleanup import CobolCleanup
from idms_db2_converter.transformers.field_rewriter import FieldRewriter
from idms_db2_converter.transformers.paragraph_rewriter import
ParagraphRewriter
from idms_db2_converter.transformers.operation_graph_transformer import
OperationGraphTransformer


class RetrievalTransformer:
    def __init__(self, schema: SchemaModel, analysis: CobolAnalysis):
        self.schema = schema
        self.analysis = analysis

    def transform(self, cobol: str, target_program: str | None = None) ->
str:
        text = cobol

        if target_program and self.analysis.program_id:
            text = self._rename_program(text, self.analysis.program_id,
target_program)

        text = self._remove_idms_control_section(text)
        text = self._remove_schema_section(text)
        text = self._remove_idms_copybooks(text)

        text = self._insert_sql_declarations(text)
        text = self._replace_init_bind_ready(text)

        operation_graph_transformer =
OperationGraphTransformer(self.schema)
```

```
        if operation_graph_transformer.has_graph():
            text = operation_graph_transformer.transform(text)
            used_cursor_sets =
sorted(operation_graph_transformer.used_cursor_sets)
        else:
            paragraph_rewriter = ParagraphRewriter(self.schema)
            text = paragraph_rewriter.rewrite(text)
            used_cursor_sets =
sorted(paragraph_rewriter.controlled_loop_sets)

        text = FieldRewriter(self.schema).rewrite(text)

        text = self._insert_fetch_paragraphs(
            text=text,
            used_cursor_sets=used_cursor_sets,
        )

        text = self._replace_idms_status_perform(text)
        text = self._replace_finish(text)
        text = self._clean_end_processing(text)
        text = self._remove_idms_abort_paragraphs(text)
        text = self._remove_obsolete_set_walk_guard(text)
        text = self._normalize_formatting(text)
        text = CobolCleanup().clean(text)
        text = self._add_sql_error_paragraph(text)

        return text

    def _rename_program(self, text: str, source: str, target: str) -> str:
        return re.sub(
            rf"(PROGRAM-ID\.\s+){re.escape(source)}(\.)",
            rf"\1{target}\2",
            text,
            flags=re.IGNORECASE,
        )

    def _remove_idms_control_section(self, text: str) -> str:
        return re.sub(
            r"\n\s*IDMS-CONTROL SECTION\..*?(?=\n\s*DATA DIVISION\.)",
```

```
            "\n",
            text,
            flags=re.IGNORECASE | re.DOTALL,
        )

    def _remove_schema_section(self, text: str) -> str:
        return re.sub(
            r"\n\s*SCHEMA SECTION\..*?(?=\n\s*FILE SECTION\.)",
            "\n",
            text,
            flags=re.IGNORECASE | re.DOTALL,
        )

    def _remove_idms_copybooks(self, text: str) -> str:
        return re.sub(
            r"^\s*(01\s+)?COPY\s+IDMS\s+.*?\.\s*$",
            "",
            text,
            flags=re.IGNORECASE | re.MULTILINE,
        )

    def _insert_sql_declarations(self, text: str) -> str:
        used_records = self._used_records()
        used_cursor_sets = self._used_declared_cursor_sets()

        host_vars = HostVariableGenerator().generate(self.schema,
used_records)
        cursors = CursorGenerator().generate(self.schema,
used_cursor_sets)

        block = "\n".join(
            [
                "",
                "
*****************************************************************",
                "    * DB2 SQLCA, HOST VARIABLES, AND CURSORS",
                "
*****************************************************************",
                host_vars,
                "",
```

```
                cursors,
                "",
            ]
        )

        return re.sub(
            r"(?=\n\s*PROCEDURE DIVISION\.)",
            block,
            text,
            flags=re.IGNORECASE,
        )

    def _insert_fetch_paragraphs(self, text: str, used_cursor_sets:
list[str]) -> str:
        if not used_cursor_sets:
            return text

        existing = {
            item.upper()
            for item in re.findall(
                r"^\s*(FETCH-[A-Z0-9-]+)\.",
                text,
                flags=re.IGNORECASE | re.MULTILINE,
            )
        }

        missing_sets = [
            set_name
            for set_name in used_cursor_sets
            if f"FETCH-{set_name}".upper() not in existing
        ]

        if not missing_sets:
            return text

        fetch_paragraphs =
FetchParagraphGenerator(self.schema).generate(missing_sets)

        if not fetch_paragraphs.strip():
            return text
```

```
        return re.sub(
            r"(?=\n\s*MAIN-LINE\.)",
            "\n" + fetch_paragraphs + "\n",
            text,
            flags=re.IGNORECASE,
        )

    def _replace_init_bind_ready(self, text: str) -> str:
        return re.sub(
            r"(\n\s*INIT-BIND-READY\.\n).*?(?=\n\s*INIT-FILES\.)",
            "\n      INIT-BIND-READY.\n"
            "
*****************************************************************\n"
            "     * IDMS BIND AND READY REMOVED FOR DB2.\n"
            "
*****************************************************************\n"
            "       CONTINUE.\n",
            text,
            flags=re.IGNORECASE | re.DOTALL,
        )

    def _replace_idms_status_perform(self, text: str) -> str:
        return re.sub(
            r"^\s*PERFORM\s+IDMS-STATUS\.?\s*$",
            "       IF SQLCODE NOT = 0 AND SQLCODE NOT = 100\n"
            "          PERFORM SQL-ERROR\n"
            "       END-IF.",
            text,
            flags=re.IGNORECASE | re.MULTILINE,
        )

    def _replace_finish(self, text: str) -> str:
        return re.sub(
            r"^\s*FINISH\.\s*$",
            "       CONTINUE.",
            text,
            flags=re.IGNORECASE | re.MULTILINE,
        )
```

```
    def _clean_end_processing(self, text: str) -> str:
        text = re.sub(
            r"(\n\s*END-PROCESSING\.\n)"
            r"\s*CONTINUE\.\s*"
            r"(?:\n\s*IF SQLCODE NOT = 0 AND SQLCODE NOT = 100\s*"
            r"\n\s*PERFORM SQL-ERROR\s*"
            r"\n\s*END-IF\.)?",
            r"\1",
            text,
            flags=re.IGNORECASE,
        )

        return text

    def _remove_idms_abort_paragraphs(self, text: str) -> str:
        return re.sub(
            r"\n\s*IDMS-ABORT\.\s*"
            r"(?:\n\s*EXIT\.)?"
            r"\s*\n\s*IDMS-ABORT-EXIT\.\s*",
            "\n",
            text,
            flags=re.IGNORECASE | re.DOTALL,
        )

    def _remove_obsolete_set_walk_guard(self, text: str) -> str:
        return re.sub(
            r"\n\s*CONTINUE\.\s*"
            r"\n\s*IF SQLCODE = 100\s*"
            r"\n\s*GO TO [A-Z0-9-]+-EXIT\s*"
            r"\n\s*ELSE\s*"
            r"\n\s*IF SQLCODE NOT = 0 AND SQLCODE NOT = 100\s*"
            r"\n\s*PERFORM SQL-ERROR\s*"
            r"\n\s*END-IF\.",
            "\n       CONTINUE.",
            text,
            flags=re.IGNORECASE,
        )

    def _normalize_formatting(self, text: str) -> str:
        text = re.sub(
```

```
            r"\n\s*CLOSE DEPT-FILE-OUT\.",
            "\n       CLOSE DEPT-FILE-OUT.",
            text,
            flags=re.IGNORECASE,
        )

        text = re.sub(
            r"\n\s+MOVE HV-",
            "\n       MOVE HV-",
            text,
            flags=re.IGNORECASE,
        )

        return text

    def _add_sql_error_paragraph(self, text: str) -> str:
        if "SQL-ERROR." in text.upper():
            return text

        return text.rstrip() + """

     SQL-ERROR.
       MOVE SPACES TO ERR-LINE.
       MOVE 'SQL ERROR' TO ERR-MESS-OUT.
       MOVE ERR-DETAIL-LINE TO ERR-LINE.
       PERFORM U200-WRITE-ERR-LINE.

     SQL-ERROR-EXIT.
       EXIT.
"""

    def _used_records(self) -> list[str]:
        records = set(self.analysis.idms_records)

        for record in self.analysis.obtain_calc_records:
            records.add(record)

        for record, _ in self.analysis.obtain_next:
            records.add(record)
```

```
        for set_name in self._used_all_sets():
            rel = self.schema.relationships.get(set_name)
            if rel:
                records.add(rel.parent_record)
                records.add(rel.child_record)

        return sorted(records)

    def _used_declared_cursor_sets(self) -> list[str]:
        sets = set()

        for _, set_name in self.analysis.obtain_next:
            sets.add(set_name)

        return sorted(sets)

    def _used_all_sets(self) -> list[str]:
        sets = set()

        for _, set_name in self.analysis.obtain_next:
            sets.add(set_name)

        for set_name in self.analysis.find_first_sets:
            sets.add(set_name)

        for set_name in self.analysis.obtain_owner_sets:
            sets.add(set_name)

        return sorted(sets)
```

src\idms_db2_converter\transformers\structured_blocks.py

```
class StructuredBlocks:
    @staticmethod
    def indent(block: str, spaces: int) -> str:
        pad = " " * spaces
        lines = []

        for line in block.splitlines():
            stripped = line.strip()
            if stripped:
                lines.append(pad + stripped)
            else:
                lines.append("")

        return "\n".join(lines)

    @staticmethod
    def sqlcode_select_block(
        sql: str,
        not_found_body: str,
        success_body: str,
        indent: int = 7,
    ) -> str:
        pad = " " * indent
        inner = " " * (indent + 3)
        inner2 = " " * (indent + 6)

        return "\n".join(
            [
                StructuredBlocks.indent(sql, indent),
                "",
                f"{pad}IF SQLCODE = 100",
                StructuredBlocks.indent(not_found_body, indent + 3),
                f"{pad}ELSE",
                f"{inner}IF SQLCODE NOT = 0",
                f"{inner2}PERFORM SQL-ERROR",
                f"{inner}ELSE",
                StructuredBlocks.indent(success_body, indent + 6),
                f"{inner}END-IF",
```

```
                f"{pad}END-IF.",
            ]
        )

    @staticmethod
    def select_first_then_owner_chain(
        first_select_sql: str,
        owner_select_sql: str,
        first_not_found_body: str,
        owner_not_found_body: str,
        success_body: str,
        indent: int = 7,
    ) -> str:
        pad = " " * indent
        inner = " " * (indent + 3)
        inner2 = " " * (indent + 6)
        inner3 = " " * (indent + 9)

        return "\n".join(
            [
                StructuredBlocks.indent(first_select_sql, indent),
                "",
                f"{pad}IF SQLCODE = 100",
                StructuredBlocks.indent(first_not_found_body, indent + 3),
                f"{pad}ELSE",
                f"{inner}IF SQLCODE NOT = 0",
                f"{inner2}PERFORM SQL-ERROR",
                f"{inner}ELSE",
                StructuredBlocks.indent(owner_select_sql, indent + 6),
                "",
                f"{inner2}IF SQLCODE = 100",
                StructuredBlocks.indent(owner_not_found_body, indent + 9),
                f"{inner2}ELSE",
                f"{inner3}IF SQLCODE NOT = 0",
                f"{inner3}   PERFORM SQL-ERROR",
                f"{inner3}ELSE",
                StructuredBlocks.indent(success_body, indent + 12),
                f"{inner3}END-IF",
                f"{inner2}END-IF",
                f"{inner}END-IF",
```

```
                f"{pad}END-IF.",
            ]
        )

    @staticmethod
    def nullable_owner_block(
        null_indicator: str,
        null_body: str,
        owner_select_sql: str,
        success_body: str,
        fallback_body: str,
        indent: int = 7,
    ) -> str:
        pad = " " * indent
        inner = " " * (indent + 3)

        return "\n".join(
            [
                f"{pad}IF {null_indicator} < 0",
                StructuredBlocks.indent(null_body, indent + 3),
                f"{pad}ELSE",
                StructuredBlocks.indent(owner_select_sql, indent + 3),
                "",
                f"{inner}IF SQLCODE = 0",
                StructuredBlocks.indent(success_body, indent + 6),
                f"{inner}ELSE",
                StructuredBlocks.indent(fallback_body, indent + 6),
                f"{inner}END-IF",
                f"{pad}END-IF.",
            ]
        )

    @staticmethod
    def owner_select_block(
        owner_select_sql: str,
        success_body: str,
        fallback_body: str,
        indent: int = 7,
    ) -> str:
        pad = " " * indent
```

```
        inner = " " * (indent + 3)

        return "\n".join(
            [
                StructuredBlocks.indent(owner_select_sql, indent),
                "",
                f"{pad}IF SQLCODE = 0",
                StructuredBlocks.indent(success_body, indent + 3),
                f"{pad}ELSE",
                StructuredBlocks.indent(fallback_body, indent + 3),
                f"{pad}END-IF.",
            ]
        )

    @staticmethod
    def cursor_loop_block(
        cursor_name: str,
        fetch_paragraph: str,
        empty_body: str,
        before_loop_body: str,
        walk_paragraph: str,
        walk_exit: str,
        indent: int = 7,
    ) -> str:
        pad = " " * indent
        inner = " " * (indent + 3)
        inner2 = " " * (indent + 6)

        return "\n".join(
            [
                f"{pad}EXEC SQL",
                f"{pad}     OPEN {cursor_name}",
                f"{pad}END-EXEC.",
                "",
                f"{pad}IF SQLCODE NOT = 0",
                f"{inner}PERFORM SQL-ERROR",
                f"{pad}ELSE",
                f"{inner}PERFORM {fetch_paragraph}",
                "",
                f"{inner}IF SQLCODE = 100",
```

```
                StructuredBlocks.indent(empty_body, indent + 6),
                f"{inner}ELSE",
                StructuredBlocks.indent(before_loop_body, indent + 6),
                "",
                f"{inner2}PERFORM UNTIL SQLCODE = 100",
                f"{inner2}    PERFORM {walk_paragraph} THRU {walk_exit}",
                f"{inner2}    PERFORM {fetch_paragraph}",
                f"{inner2}END-PERFORM",
                f"{inner}END-IF",
                "",
                f"{inner}EXEC SQL",
                f"{inner}     CLOSE {cursor_name}",
                f"{inner}END-EXEC",
                f"{pad}END-IF",
            ]
        )
```

src\idms_db2_converter\validators\output_validator.py

```
import re


class OutputValidator:
    FORBIDDEN = [
        r"\bOBTAIN\b",
        r"\bFIND\b",
        r"\bREADY\b",
        r"\bBIND\b",
        r"\bFINISH\b",
        r"\bKEEP\b",
        r"\bERASE\b",
    ]

    REQUIRED = [
        "EXEC SQL",
        "SQLCA",
        "END-EXEC",
    ]

    DECLARED_CURSOR_PATTERN = re.compile(
        r"DECLARE\s+(C-[A-Z0-9-]+)\s+CURSOR",
        re.IGNORECASE,
    )

    OPEN_CURSOR_PATTERN = re.compile(
        r"OPEN\s+(C-[A-Z0-9-]+)",
        re.IGNORECASE,
    )

    FETCH_CURSOR_PATTERN = re.compile(
        r"FETCH\s+(C-[A-Z0-9-]+)",
        re.IGNORECASE,
    )

    CLOSE_CURSOR_PATTERN = re.compile(
        r"CLOSE\s+(C-[A-Z0-9-]+)",
        re.IGNORECASE,
```

```
    )

    def validate(self, cobol: str) -> list[str]:
        errors = []
        executable_text = self._remove_comment_lines(cobol)
        upper = executable_text.upper()

        for pattern in self.FORBIDDEN:
            if re.search(pattern, upper, flags=re.IGNORECASE |
re.MULTILINE):
                errors.append(f"Forbidden IDMS pattern remains:
{pattern}")

        for token in self.REQUIRED:
            if token not in upper:
                errors.append(f"Required DB2 token missing: {token}")

        self._validate_no_nested_host_variables(upper, errors)
        self._validate_no_rewritten_report_fields(upper, errors)
        self._validate_no_idms_schema_fields(upper, errors)
        self._validate_sql_punctuation(upper, errors)
        self._validate_cursor_lifecycle(upper, errors)

        return errors

    def _remove_comment_lines(self, cobol: str) -> str:
        lines = []

        for line in cobol.splitlines():
            stripped = line.strip()

            if stripped.startswith("*"):
                continue

            lines.append(line)

        return "\n".join(lines)

    def _validate_no_nested_host_variables(
        self,
```

```
        upper: str,
        errors: list[str],
    ) -> None:
        if re.search(r"\bHV-[A-Z0-9-]+-HV-[A-Z0-9-]+\b", upper):
            errors.append(
                "Converted output contains nested host variables like
HV-...-HV-.... "
                "Field rewriting is being applied to generated host
variables."
            )

    def _validate_no_rewritten_report_fields(
        self,
        upper: str,
        errors: list[str],
    ) -> None:
        bad_patterns = [
            r"\bHV-[A-Z0-9-]+-IN\b",
            r"\bHV-[A-Z0-9-]+-OUT\b",
        ]

        for pattern in bad_patterns:
            if re.search(pattern, upper):
                errors.append(
                    "Converted output appears to have rewritten
input/output report fields. "
                    "Field rewriting should only affect IDMS record
fields, not file/report fields."
                )
                return

    def _validate_no_idms_schema_fields(
        self,
        upper: str,
        errors: list[str],
    ) -> None:
        if re.search(r"\b[A-Z0-9]+(?:-[A-Z0-9]+)*-\d{4}\b", upper):
            errors.append(
                "Converted output still contains IDMS schema-suffixed
fields like FIELD-0415."
```

```
            )

    def _validate_sql_punctuation(
        self,
        upper: str,
        errors: list[str],
    ) -> None:
        if ", \nFROM" in upper or ",\nFROM" in upper:
            errors.append("Trailing comma found before FROM.")

        if ", \nEND-EXEC" in upper or ",\nEND-EXEC" in upper:
            errors.append("Trailing comma found before END-EXEC.")

        if re.search(
            r"SELECT\s+[A-Z0-9_]+\n\s+[A-Z0-9_]+\n",
            upper,
            flags=re.IGNORECASE,
        ):
            errors.append("SQL SELECT list may be missing commas.")

    def _validate_cursor_lifecycle(
        self,
        upper: str,
        errors: list[str],
    ) -> None:
        declared = set(self.DECLARED_CURSOR_PATTERN.findall(upper))
        opened = set(self.OPEN_CURSOR_PATTERN.findall(upper))
        fetched = set(self.FETCH_CURSOR_PATTERN.findall(upper))
        closed = set(self.CLOSE_CURSOR_PATTERN.findall(upper))

        for cursor in declared:
            if cursor not in fetched:
                errors.append(f"Cursor {cursor} is declared but never
fetched.")

            if cursor not in opened:
                errors.append(f"Cursor {cursor} is declared but never
opened.")

            if cursor not in closed:
```

```
                errors.append(f"Cursor {cursor} is declared but never
closed.")
```

src\idms_db2_converter\validators\schema_validator.py

```
from idms_db2_converter.models import SchemaModel, CobolAnalysis


class SchemaValidator:
    def validate(self, schema: SchemaModel, analysis: CobolAnalysis) ->
list[str]:
        errors = []

        for record in analysis.idms_records:
            if record not in schema.records:
                errors.append(f"COBOL references record {record}, but
schema does not define it.")

        for record in analysis.obtain_calc_records:
            if record not in schema.records:
                errors.append(f"OBTAIN CALC references missing record
{record}.")
            elif not schema.records[record].primary_key:
                errors.append(f"OBTAIN CALC record {record} has no primary
key.")

        for _, set_name in analysis.obtain_next:
            self._validate_set(schema, set_name, errors)

        for set_name in analysis.obtain_owner_sets:
            self._validate_set(schema, set_name, errors)

        for set_name in analysis.find_first_sets:
            self._validate_set(schema, set_name, errors)

        return errors

    def _validate_set(
        self,
        schema: SchemaModel,
        set_name: str,
        errors: list[str],
```

```
    ) -> None:
        if set_name not in schema.relationships:
            errors.append(f"COBOL references set {set_name}, but schema
relationships do not define it.")
            return

        rel = schema.relationships[set_name]

        if not rel.parent_key:
            errors.append(f"Set {set_name} has no parent_key.")

        if not rel.child_fk:
            errors.append(f"Set {set_name} has no child_fk.")

        if rel.parent_record not in schema.records:
            errors.append(f"Set {set_name} parent record
{rel.parent_record} is missing.")

        if rel.child_record not in schema.records:
            errors.append(f"Set {set_name} child record {rel.child_record}
is missing.")
```